

Data Adaptive Continuous Release for Infinite Streams under Local Differential Privacy

Xiaojie Cao¹, Yinbin Miao^{1,§}, XXX², XXX²

¹Xidian University, Xi'an, China ²XX, XX, China

³XX, XX, China

24151111248@stu.xidian.edu.cn, ybmiao@xidian.edu.cn, XXX, XXX

Abstract—Real-time data plays a crucial role in various scenarios, such as health monitoring. However, directly collecting and analyzing such data may leak users' sensitive information. To address this, the w -event local differential privacy paradigm has become the standard for data stream collection and analysis, as it allows private data to be perturbed directly on the user side. Nevertheless, existing methods for publishing data streams still suffer from issues such as inefficient use of the privacy budget and an inability to predict stream trends based on historical releases. To overcome these limitations, we propose a basic data-adaptive publishing mechanism called Flexible Budget Absorption (Fle-BA). Fle-BA selects valuable data for release based on fluctuations in the data stream, effectively improving the utilization of the privacy budget. Furthermore, to fully extract data features within the sliding window, we introduce an enhanced method, Adaptive Grouping Smoothing (AGS), which groups data based on their similarity and then performs post-processing on the resulting groups. Comprehensive experimental results demonstrate that our proposed methods deliver superior performance over multiple real-world and synthetic datasets, achieving an average 10× improvement in accuracy compared to state-of-the-art stream publishing solutions.

Index Terms—Local differential privacy, w -event privacy, data streams

I. INTRODUCTION

In the age of big data, systems such as the Internet of Things (IoT) and remote healthcare continuously generate massive volumes of data. Efficient collection and analysis of these real-time data streams have become essential to modern distributed infrastructure. To extract real-time insights from this high-velocity data, continuous monitoring, aggregation, and statistical analysis are essential [1]–[3]. However, such data usually contains highly sensitive individual information, and directly processing or releasing statistics may leak personal privacy. *Local Differential Privacy* (LDP) [4], [5] is a widely accepted framework for the collection and analysis of sensitive data. As it does not require a trusted third party, LDP has been widely deployed in real-world systems, including Google [6], Apple [7] and Microsoft [8].

In privacy-preserving continuous data collection and release, two main privacy notions are commonly considered: event-level privacy [9], [10] and user-level privacy [11], [12]. The former protects only individual events and cannot prevent adversaries from inferring users' long-term behavioral patterns. The latter protects all data values contributed by a user

throughout the entire stream, but it is typically applies only to finite data streams. The emergence of w -event privacy [13] bridges the gap between event-level and user-level privacy. By protecting any sequence occurring within any window of w timestamps, w -event privacy offers stronger practicality in continuous data release scenarios. Several works [14]–[17] have been proposed to publish infinite data streams under w -event LDP. These works focus on releasing data streams that ensure privacy while maintaining optimal utility. However, there are still two key limitations.

First, existing mechanisms fail to use the privacy budget efficiently, making it difficult to balance estimation error with the information released. LDP-IDS [14], a representative method, decides whether to release the current data based on its deviation from historical data. If the current data is not published, the mechanism transfers the corresponding budget to subsequent timestamps and forces the skipping of an equal-length time interval. Although this blind skipping strategy ensures that the window budget does not exceed ϵ , it ignores the dynamic characteristics of the real data stream. In real scenarios, future data patterns are uncertain, and the selected published data are often the key information points. However, the fixed equal-length skipping mechanism leads to missed release opportunities and the loss of critical information. Moreover, if too much data is published within any window, the budget of each timestamp will be diluted, thereby introducing excessive noise [15], [16].

Second, existing mechanisms lack generality and cannot effectively predict future data trends. In a data stream, data at each timestamp is not isolated but exhibits temporal correlations. Therefore, existing methods [12], [16] leverage the fact that each local user can obtain both real and noisy data. CGM [12] reduces noise by perturbing the deviation between adjacent data points, but it provides a relaxed privacy guarantee and is suitable for finite streams. As a state-of-the-art data stream publishing method, CAPP [16] uses cumulative data deviations to calibrate the noise error. However, the underlying design of these methods relies on numerical data, making them inapplicable for processing discrete data at the local user side. Moreover, they are unable to accurately capture changes in data stream trends.

To overcome the above issues, we offer corresponding methods. Specifically, to trade off the released information and estimation error, we consider selecting a fixed number

[§] Yinbin Miao is the corresponding author.

of timestamps within each sliding window for data release. Nevertheless, predefined sample rates are difficult to adapt to changes in data streams. To remedy the deficiency of existing methods, we propose the Flexible Budget Absorption (Fle-BA) algorithm, which samples valuable data for release while ensuring that the total privacy budget consumed within any sliding window does not exceed ϵ . Fle-BA constructs a private dissimilarity function between statistical results at adjacent timestamps to predict trend changes in the data stream, and uses this function to determine whether to publish data at the current timestamp. The privacy budget of unpublished data will be preserved and reallocated for subsequent data releases. This method enables dynamic accumulation and flexible transfer of unused privacy budgets, effectively avoiding insufficient utilization of the privacy budget caused by excessive skipping in traditional methods.

To make full use of timestamp data and steadily improve the accuracy of statistical results under strict privacy constraints, we propose the Adaptive Grouping Smoothing (AGS) method, which introduces a novel perspective on data grouping and post-processing optimization. AGS constructs an adaptive grouping function to measure data similarity and uses it to dynamically group statistical results for each timestamp. During grouping, we record not only the overall grouping results but also the partition information for each timestamp. To enhance utility, AGS performs post-processing based on the grouping and partitioning results, leveraging multiple historical data points within the same group to optimize the current statistical value. We conduct comprehensive experiments on five real-world datasets and three synthetic datasets, and the results validate the effectiveness of our proposed approaches.

In summary, we make the following contributions:

- We design the adaptive publishing function and adaptive grouping function to achieve accurate and efficient statistical analysis of noisy streams, while strictly ensuring that continuous publication satisfies w -event ϵ -LDP guarantee.
- We propose Fle-BA, which flexibly reallocates unused privacy budgets in response to data variations, thereby minimizing information loss while guaranteeing sliding window budget constraints. We release valuable data based on the private dissimilarity function.
- We consider data grouping under LDP framework to overcome the accuracy limitations in budget allocation. Therefore, we propose AGS, which adaptively groups perturbed data and applies post-processing smoothing techniques to effectively reduce noise.
- We conduct comprehensive experimental evaluations on real world and synthetic datasets. The results demonstrate our approaches consistently achieve a $10 \times$ improvement in accuracy under the same parameter settings.

The remainder of this paper is organized as follows. In Section II, we review the related work. In Sections III and IV, we introduce some preliminaries and problem formulation, respectively. In Sections V and VI, we detail the proposed methods. In Sections VII and VIII, we conduct an extended

discussion and provide a theoretical analysis of all methods. In Section IX, we describe the experimental results. Finally, we conclude the paper in Section X.

II. RELATED WORK

CDP [18], [19] and LDP [4], [5] have been widely applied to various data analysis tasks, including frequency estimation [6], [20]–[22], mean estimation [23], [24] and data mining [25], [26]. In the context of continuous data publication, privacy-preserving research has mainly focused on three aspects: event-level privacy, user-level privacy and w -event privacy.

Event-level Privacy. Event-level protects the privacy of a single event in a continuous data stream [11], [27]–[29]. TopL [27] determines a truncation threshold by minimizing the overall error and subsequently perturbs data at each timestamp. However, this method severely degrades the utility of released data. To enhance data accuracy, PeGaSuS [28] smooths noisy data by leveraging similar-characteristic data, but it suffers from suboptimal utilization of data during the post-processing phase. Consequently, Li *et al.* [29] propose a novel delay-allowed data-releasing framework. By employing group-based and order-based strategies within delay windows, this framework significantly reduces noise scale while maintaining ϵ -DP.

User-level Privacy. User-level protects the privacy of all the events of any user throughout the entire data stream, existing works include CGM [12], PrivShape [30] and DPI [31]. Bao *et al.* propose a Correlated Gaussian Mechanism (CGM) [12], which ingeniously leverages temporal correlations in the data stream to reduce overall noise. However, CGM is restricted to finite data streams and only satisfies approximate LDP, i.e., (ϵ, δ) -LDP. Mao *et al.* introduce PrivShape [30], a prefix tree-based method designed for shape extraction in time-series data, but it performs poorly in estimating the mean and distribution of range queries. DPI [31] effectively bounds privacy leakage while enhancing the analytical accuracy of real-time queries over infinite data streams.

Event-level privacy struggles to meet users' coarse-grained privacy requirements, and user-level privacy is difficult to implement on infinite data streams. To overcome this dilemma, w -event privacy for infinite data streams is proposed [13], which protects any event sequence occurring within any window of w timestamps.

w -event Privacy. Several existing works have been proposed [14]–[17], [32]–[35] to publish infinite streams. SPAS [34] continuously predicts an optimal publishing strategy within each sliding window, which minimizes release error based on the characteristics of data stream. PatternLDP [17] samples and perturbs only those points in a time series that have a significant impact on the pattern, thereby preserving essential pattern characteristics while ensuring privacy. RetraSyn [15] is a real-time trajectory stream synthesis framework under LDP, which captures complex movement patterns to construct global mobility model, enabling the dynamic release and synthesis of real-time trajectory streams. As the first w -event privacy framework to achieve ϵ -LDP on infinite data streams, LDP-IDS [14] proposes two classes of methods. One is an

TABLE I
COMPARISON SUMMARY OF EXISTING SOLUTIONS AND OUR SCHEMES.

Solutions	Publication Strategy	Data Type	Budget Allocation Flexibility	Multi-point Historical Dependency	Trend Prediction	Privacy Level
PatternLDP [17]	Sample	Numerical	●	×	×	w -event
PrivShape [30]	Sample	Categorical	○	×	×	user-level
LBA [14]	Sample	Universal	●	×	✓	w -event
CGM [12]	Uniform	Numerical	○	×	×	user-level
RetraSyn [15]	Uniform	Categorical	○	×	✓	w -event
CAPP [16]	Uniform	Numerical	○	✓	×	w -event
Fle-BA (Ours)	Sample	Universal	●	×	✓	w -event
AGS (Ours)	Uniform	Universal	○	✓	✓	w -event

Note: ● indicates flexible privacy budget adjustment based on released data and efficient budget utilization; ○ indicates no dynamic budget adjustment; ● indicates a relatively rigid budget adjustment strategy with limited budget utilization.

adaptive budget division method that effectively improves data utility; the other is an adaptive population division method that achieves higher utility with lower communication overhead. CAPP [16] uses previous noise to dynamically calibrate subsequent values, and applies interval clipping and Simple Moving Average (SMA) smoothing, which enhances data utility.

III. PRELIMINARIES

In this section, we introduce the knowledge of local differential privacy, w -event privacy and frequency oracles.

A. Local Differential Privacy

Local differential privacy (LDP) [4], [5] allows each user independently perturb their data and then report the perturbed output to the server for aggregation. The ϵ -LDP notion is formulated as follows:

Definition 1 (Local Differential Privacy [4]). *A random algorithm $\mathcal{M}$ satisfies ϵ -local differential privacy, if and only if for any two values v, v' and all possible output $y \subseteq \text{Range}(\mathcal{M})$, the Equation. 1 holds*

$$\Pr[\mathcal{M}(v) = y] \leq e^\epsilon \times \Pr[\mathcal{M}(v') = y]. \quad (1)$$

Here, ϵ denotes the privacy budget, which quantifies the level of privacy guarantee. A smaller ϵ means that the outputs of $\mathcal{M}$ on two input values are more similar, thereby providing a stronger privacy guarantee. To make the entire process adhere to the requirements of LDP, we rely on the sequential composition property, which guarantees the overall privacy bound for a sequence of randomized algorithms.

Theorem 3.1 (Sequential Composition [36]). *Given a set of k randomized algorithms $\{\mathcal{M}_1, \mathcal{M}_2, \dots, \mathcal{M}_k\}$, each algorithm $\mathcal{M}_i (1 \leq i \leq k)$ satisfies ϵ_i -LDP. Then, the sequential composition of these algorithms satisfies ϵ -LDP, where $\epsilon = \sum_{i=1}^k \epsilon_i$.*

Theorem 3.2 (Post-processing). *Given an ϵ -LDP algorithm $\mathcal{M}$ and an arbitrary randomized mapping function g . Then, $g(\mathcal{M}(\cdot))$ still satisfies ϵ -LDP.*

The post-processing procedure only accesses the privatized outputs generated by $\mathcal{M}$, rather than the original sensitive data.

That is, post-processing the output of an LDP algorithm does not incur any additional privacy loss.

B. w -Event Local Differential Privacy

w -event privacy [13] is an LDP model tailored for continuous data streams. It strikes a nice privacy balance between event-level and user-level privacy by protecting arbitrary w consecutive timestamps in a data stream. We consider $V = \{V[1], V[2], \dots\}$ represents a user's infinite data stream. The stream prefix V_t of V is defined as $V_t = \{V[1], V[2], \dots, V[t]\}$, which represents the sequence of values up to timestamp t . Then, the notion of neighboring stream prefixes is defined as follows.

Definition 2 (w -neighboring). *For a positive integer w , two stream prefixes V_t, V'_t are w -neighboring, if*

- (i) *for each $V_t[i], V'_t[i]$ such that $V_t[i] \neq V'_t[i]$, it holds that $V_t[i], V'_t[i]$ are neighboring, and*
- (ii) *for each $V_t[i_1], V'_t[i_1], V_t[i_2], V'_t[i_2]$ with $i_1 < i_2$, $V_t[i_1] \neq V'_t[i_1]$ and $V_t[i_2] \neq V'_t[i_2]$, it holds that $i_2 - i_1 + 1 \leq w$.*

Two data streams are w -neighboring means that all pairwise unequal values can fit in a window of up to w timestamps.

Definition 3 (w -Event ϵ -LDP). *Let $\mathcal{M}$ be a mechanism that takes as input a stream of arbitrary size, and O be the set of all possible outputs of $\mathcal{M}$. $\mathcal{M}$ satisfies w -event ϵ -LDP if for any w -neighboring stream V_t and V'_t and all t , the Equation 2 holds*

$$\Pr[\mathcal{M}(V_t) = O] \leq e^\epsilon \times \Pr[\mathcal{M}(V'_t) = O]. \quad (2)$$

w -event ϵ -local differential privacy characterizes sensitive information originating from an event sequence of length w , i.e. from a set of events that (i) each event occurs at a distinct timestamp and (ii) all events occur within w consecutive timestamps. For example, in a traffic data release service, an event sequence of length w might represent the mobility trajectory of a commuter over w consecutive timestamps. In this setting, the primary objective of w -event privacy is to protect the trajectory information for any commuter within any such temporal window.

To systematically design mechanisms that satisfy w -event ϵ -local differential privacy, we adopt a sliding-window framework. The core idea is to decompose the overall mechanism $\mathcal{M}$ into a sequence of sub-mechanisms $\mathcal{M} = \{\mathcal{M}_1, \mathcal{M}_2, \dots\}$, where each $\mathcal{M}_t$ operates solely on the data item at timestamp t . If each sub-mechanism $\mathcal{M}_t$ satisfies ϵ_t -LDP, to ensure the overall mechanism $\mathcal{M}$ satisfies w -event privacy only requires the condition: for any timestamp t , the active window of length w span the interval from $t-w+1$ to t , then $\sum_{i=t-w+1}^t \epsilon_i \leq \epsilon$. As long as the cumulative privacy budget within arbitrary w consecutive timestamps is bounded by ϵ , the overall mechanism $\mathcal{M}$ guarantees w -event ϵ -LDP. In Theorem 8.1, we provide the privacy proof.

C. Frequency Oracles

Under the LDP framework, perturbation mechanisms for categorical data are commonly referred to as frequency oracle (FO) protocols [22]. An FO protocol enables unbiased estimation of the frequency of any value v in a domain D of size $d = |D|$. Below, we detail two classical FO protocols.

GRR (Generalized Randomized Response) [22], [37] is a generalized version the randomized response technique [38]. Each user locally reports the true value v with a higher probability $p = \frac{e^\epsilon}{e^\epsilon + d - 1}$, and reports any other value $v' \in D$ s.t. $v' \neq v$ with a lower probability $q = \frac{1}{e^\epsilon + d - 1}$. After receiving the perturbed reports from N users, the server estimates the frequency of each category. For a given value v , the estimated result of its true frequency $f(v)$ can be calculated as $\hat{f}(v) = \frac{f'(v) - q}{p - q}$, where $f'(v)$ denotes the number of occurrences of value v in all reports. In [22], it is shown that $\hat{f}(v)$ is an unbiased estimation, and the variance is

$$\text{Var}[\hat{f}_{\text{GRR}}(v)] = \frac{d - 2 + e^\epsilon}{N(e^\epsilon - 1)^2}. \quad (3)$$

Optimized Unary Encoding (OUE) [22] encodes an input value $v \in D$ as a length- d binary vector $\mathbf{B}$, where the j -th bit is 1 (index j indicates the position of value v) and all other bits are 0. The bit 1 remains 1 with probability $p = 1/2$, and 0 is flipped to 1 with probability $q = \frac{1}{e^\epsilon + 1}$ with the maximum allowed privacy budget ϵ . Given reports $\mathbf{B}_i$ from $i \in [N]$, the server to estimate the number of times that 1 at the j -th position in the reported vectors, denoted as $f'[j] = \sum \mathbb{I}_{\{j|\mathbf{B}_i[j]=1\}}$. The indicator function $\mathbb{I}(\cdot)$ outputs 1 if the inner condition is true, and 0 otherwise. Then, the unbiased estimation of true frequency of v can be derived as $\hat{f}[j] = \frac{f'[j]/N - q}{p - q}$, and the variance

$$\text{Var}[\hat{f}_{\text{OUE}}[j]] = \frac{4e^\epsilon}{N(e^\epsilon - 1)^2}. \quad (4)$$

Adaptive FO. Comparing Equation.3 with Equation.4, the factor $d - 2 + e^\epsilon$ is replaced by $4e^\epsilon$. To minimize estimation error, we adopt an Adaptive FO protocol: when the domain size d is less than $3e^\epsilon + 2$, GRR is used as FO; otherwise, OUE is used. It has variance

$$\text{Var}[\hat{f}_{\text{Fo}}[j]] = \min \left(\frac{d - 2 + e^\epsilon}{N(e^\epsilon - 1)^2}, \frac{4e^\epsilon}{N(e^\epsilon - 1)^2} \right). \quad (5)$$

To simplify mathematical derivations and maintain generality, let $\text{Var}(\epsilon, N)$ denotes the estimation variance of the adaptive FO protocol under privacy budget ϵ and N users.

IV. PROBLEM FORMALIZATION

In this section, we define the system model, problem definition and design goals.

A. System Model

In privacy-preserving real-time data collection scenarios, the system model typically involves server and local users, as shown in Figure 1.

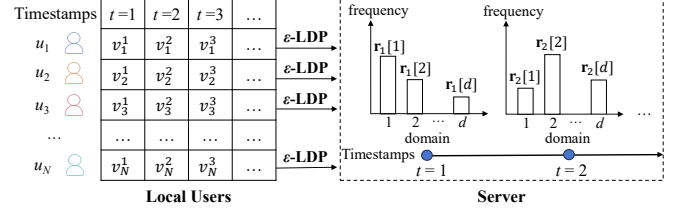


Fig. 1. System model.

Local Users. There are N users $U = \{u_1, u_2, \dots, u_N\}$, where each user u_i possesses a continuous data stream $V_i = \{v_i^1, v_i^2, \dots\}$. The value v_i^t of any timestamp belongs to a discrete value range $D = \{v_1, v_2, \dots, v_d\}$. At each timestamp, each user u_i perturbs the true data v_i^t using the adaptive FO protocol and reports the noisy value $\tilde{v}_i^t$ to the server. This local perturbation process is guaranteed to satisfy w -event ϵ -LDP.

Server. The server collects the noisy reports $\tilde{v}_i^t$ from all N users and performs frequency estimation task for any category value $v_j \in D$. Consequently, the server outputs the aggregated frequency results $\mathbf{r}_t = [\mathbf{r}_t[1], \mathbf{r}_t[2], \dots, \mathbf{r}_t[d]]$ at timestamp t , which can then be utilized for public release.

B. Problem Definition

Given a sensitive data stream $V_i = \{v_i^1, v_i^2, \dots\}$ possessed by user u_i , where any value $v_i^t \in D$ at timestamp t . All users continuously upload data, and the server aggregates the perturbed values to estimate frequencies. The frequency estimation task aims to count the occurrence frequencies of different elements. We formally define this task as follows.

Definition 4 (Frequency estimation). Let $f_t[j]$ denote the frequency of category $v_j \in D$ at timestamp t . It represents the proportion of users whose true value equals v_j , formally defined as

$$f_t[j] = \frac{1}{N} \sum_{i=1}^N \mathbb{I}(v_i^t = v_j), \quad (6)$$

where $\mathbb{I}(\cdot)$ is an indicator function that returns 1 if $v_i^t = v_j$, and returns 0 otherwise.

The server needs to release an estimated frequency sequence $\mathbf{R} = [\mathbf{r}_1, \mathbf{r}_2, \dots]$, where $\mathbf{r}_t = [\mathbf{r}_t[1], \mathbf{r}_t[2], \dots, \mathbf{r}_t[d]]$ is the noisy version of $\mathbf{f}_t = [\mathbf{f}_t[1], \mathbf{f}_t[2], \dots]$ at any timestamp t .

The primary goal is to minimize the distance between the released sequence $\mathbf{R} = [\mathbf{r}_1, \mathbf{r}_2, \dots]$ and the true sequence $\mathbf{F} = [\mathbf{f}_1, \mathbf{f}_2, \dots]$ while satisfying w -event ϵ -LDP.

Under w -event privacy, one approach selectively publishes only those data points that contain critical information [14]. For unpublished data points, their privacy budgets will be used for future data releases. For instance, within any sliding window $[i - w + 1, i]$ of length w , the privacy budget is uniformly allocated to each timestamp (i.e., $\epsilon_t = \epsilon/w$). If data is not published at timestamp t , the budget ϵ_t is saved. If data is subsequently published at timestamp $t + 1$, the privacy budget becomes $\epsilon_{t+1} = \epsilon_{t+1} + \epsilon_t$. Subsequently, the equal intervals are directly skipped, ensuring that $\sum_{t=i-w+1}^i \epsilon_t \leq \epsilon$. However, this direct skipping strategy will result in the loss of crucial information. In fact, any data that is selected for release is valuable. Moreover, under the sliding window setting, this strategy also means that the privacy budget within the window has not been maximally utilized.

Another approach leverages the correlation between data points to reduce perturbation errors through deviation feedback [16]. Based on the properties of LDP, each user knows their own true value x_t and perturbed value x'_t at timestamp t , thus the deviation $d_t = x'_t - x_t$ can be calculated. This deviation will be corrected in the next timestamp when perturbing x_{t+1} . The input value $x_{t+1}^* = x_{t+1} + d_t$ can be obtained, which is subsequently clipped to the range $[l, u]$. Finally, the SW mechanism is applied to perturb this clipped value, yielding the final noisy output x'_{t+1} . This method is difficult to be effectively applied to categorical data scenarios. The reason for this is that each user only holds a local private value at timestamp t , which makes it impossible to compute the deviation value. Moreover, this mechanism is also unable to predict the future trend of data stream.

C. Design Goals

Our designs aim to achieve three key objectives for continuous data publishing under w -event ϵ -LDP:

- **w -event privacy:** Our methods strictly satisfy w -event ϵ -LDP by ensuring that the cumulative privacy budget within any sliding window of size w remains bounded.
- **High utility:** Improve the accuracy of released results by mitigating the impact of noise and preserving useful temporal information.
- **Effectiveness:** Support continuous data publishing that sufficiently captures data evolution over time and prevent overly sparse releases.

V. OUR PROPOSED BASIC SCHEME

We first present a technical overview of our basic scheme and then provide a detailed design process.

A. Technical Overview of Fle-BA

We show the overview of Fle-BA in Figure 2. N users send perturbed data to the server via the frequency oracle for frequency estimation at each timestamp. To mitigate cumulative perturbation errors caused by information redundancy

in continuous data streams, Fle-BA employs an adaptive sampling strategy. More specifically, at each timestamp t , Fle-BA decomposes the overall mechanism $\mathcal{M}_t$ into two sub-mechanisms, i.e., private dissimilarity calculation $\mathcal{M}_{t,1}$ and private publication $\mathcal{M}_{t,2}$. Initially, the privacy budget for each timestamp is $\epsilon_{t,1} = \epsilon_{t,2} = \epsilon/2w$ in $\mathcal{M}_{t,1}$ and $\mathcal{M}_{t,2}$.

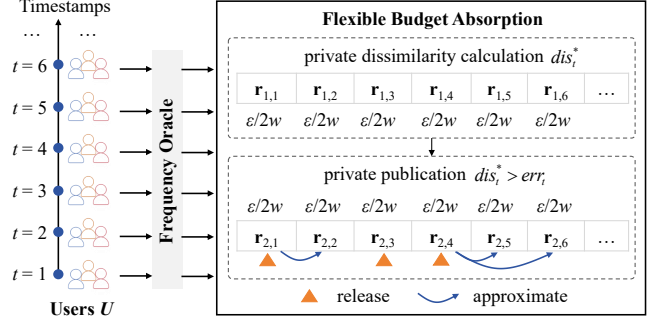


Fig. 2. The workflow of Fle-BA

- **Private Dissimilarity Calculation.** To release more representative statistics, the mechanism requires a reliable criterion. However, the server cannot access the true data, so an unbiased dissimilarity function is constructed using the perturbed data. Therefore, the sub-mechanism $\mathcal{M}_{t,1}$ is designed to extract prior information, which supports the data release of the sub-mechanism $\mathcal{M}_{t,2}$. The main challenge in $\mathcal{M}_{t,1}$ is how to privately calculate the dissimilarity dis_t^* at each timestamp.
- **Private Publication.** The mechanism $\mathcal{M}_{t,2}$ compares the dissimilarity dis_t^* with the publication error err_t to determine whether to release $\mathbf{r}_t$ at the current timestamp. If a release occurs, the budget $\epsilon_{t,2}$ is consumed; otherwise, $\mathbf{r}_t$ is approximated by the last released result $\mathbf{r}_l$ (i.e., $\mathbf{r}_t = \mathbf{r}_l$). The saved budget $\epsilon_{t,2}$ can be used for a future data release. To ensure that the accumulated budget within any sliding window strictly satisfies the ϵ constraint, we propose a flexible budget absorption strategy that reuses saved budget to maximize utilization for subsequent releases.

B. Construction of Fle-BA

We will first present the Strawman method. Then, we will introduce the details of the private dissimilarity calculation and the private publication components of the FLe-BA method.

Strawman: Multi-point Sampling. Inspired by the sampling method, we propose a *Multi-point Sampling (MPS)* method that randomly samples $m > 1$ timestamps to release statistical results within any sliding window w , allocating each timestamp a privacy budget of ϵ/m . For the remaining $w - m$ timestamps, the statistical result of the last sampled timestamp l is used as an approximation.

Example 1. Within any w -length window, at most m data points can be sampled. The sliding window size is $w = 3$, and the number of samples is $m = 2$. Within the interval $[1, 3]$, at most two time points may release statistical results. Suppose the results are published at $t = 1$ and $t = 2$, with statistical

results $\mathbf{r}_1$ and $\mathbf{r}_2$ respectively, and each point consumes a privacy budget of $\epsilon/2$. For the unsampled time point $t = 3$, its released value is approximated using the most recent released result, i.e., $\mathbf{r}_3 = \mathbf{r}_2$. When the window slides to the interval $[2, 4]$, only one additional time point may be selected for publication, ensuring that no more than two releases occur. Therefore, only $t = 4$ is sampled, and the statistical result $\mathbf{r}_4$ is published, consuming a privacy budget of $\epsilon/2$. For subsequent time points, the statistical result $\mathbf{r}_5$ is released, and $\mathbf{r}_6 = \mathbf{r}_5$ at $t = 6$.

Our approach. The fixed threshold strategy does not take into account the dynamic changes of the data itself. To achieve dynamic sampling, the ideal mechanism is to decide whether to publish based on changes in the data. We determine whether to publish data by comparing the dissimilarity between the current statistic and the last released statistics. Under LDP, the dissimilarity must be estimated from noisy statistical results. Therefore, we propose an adaptive data sampling method, namely *Flexible Budget Absorption (Fle-BA)*, that maximizes the budget utilization rate. The procedure of Fle-BA is presented in Algorithm 1.

For any sliding window of size w , the privacy budget ϵ is evenly divided into $\epsilon_1 = \epsilon/2$ for dissimilarity computation and $\epsilon_2 = \epsilon/2$ for data release. Accordingly, we decompose the mechanism $\mathcal{M}_t$ at each timestamp t into two independent sub-mechanisms $\mathcal{M}_{t,1}$ and $\mathcal{M}_{t,2}$.

Private dissimilarity calculation $\mathcal{M}_{t,1}$: We define the dissimilarity measure $dis_t = \frac{1}{d} \sum_{j=1}^d (\mathbf{f}_t[j] - \mathbf{r}_l[j])^2$, which characterizes the deviation between the true statistics $\mathbf{f}_t$ at the current time t and the last released result $\mathbf{r}_l$. Since the true statistics $\mathbf{f}_t$ is inaccessible, the dissimilarity must rely on the noisy value $\mathbf{r}_t$. To this end, the privacy budget ϵ_1 is uniformly allocated across all w timestamps for computing the dissimilarity. Specially, the server estimate the statistic value $\mathbf{r}_{t,1}$ with budget $\epsilon_{t,1} = \epsilon/2w$ at each timestamp t . Then, we can formalize the dissimilarity measure dis_t^* as follows

$$dis_t^* = \frac{1}{d} \sum_{j=1}^d (\mathbf{r}_{t,1}[j] - \mathbf{r}_l[j])^2 - \frac{1}{d} \sum_{j=1}^d \text{Var}[\mathbf{r}_{t,1}[j]], \quad (7)$$

where $\mathbf{r}_{t,1}$ denotes the unbiased estimate of $\mathbf{f}_t$ obtained via an ϵ -LDP mechanism applied to the perturbed data in $\mathcal{M}_{t,1}$. For any index $j \in [1, d]$, $\mathbb{E}[\mathbf{r}_{t,1}[j]] = \mathbf{f}_t[j]$. Thus, the expectation of dis_t^* can be expressed as

$$\begin{aligned} & \mathbb{E}[dis_t^*] \\ &= \mathbb{E} \left[\frac{1}{d} \sum_{j=1}^d (\mathbf{r}_{t,1}[j] - \mathbf{r}_l[j])^2 - \frac{1}{d} \sum_{j=1}^d \text{Var}[\mathbf{r}_{t,1}[j]] \right] \\ &= \frac{1}{d} \sum_{j=1}^d \mathbb{E}[\mathbf{r}_{t,1}[j] - \mathbf{r}_l[j]]^2 - \frac{1}{d} \sum_{j=1}^d \text{Var}[\mathbf{r}_{t,1}[j]]. \end{aligned} \quad (8)$$

For the part $\mathbb{E}[\mathbf{r}_{t,1}[j] - \mathbf{r}_l[j]]^2$ of Equation 8, we utilize the definition of variance to convert it. We express the variance of $\mathbf{r}_{t,1}[j]$ as Equation 9.

Algorithm 1: Fle-BA

Input: Data stream $\{V_1, V_2, \dots, V_N\}$, privacy budget ϵ , window size w , domain D
Output: $\mathbf{R} = [\mathbf{r}_1, \mathbf{r}_2, \dots]$

```

1 Initialize last released timestamp  $l = 0$ ;
2 for each timestamp  $t \in [i - w + 1, i]$  do
    // Sub-mechanism  $\mathcal{M}_{t,1}$ 
3 Set budget for each timestamp  $\epsilon_{t,1} = \epsilon/(2w)$ ;
4 for each user  $u_i \in U$  do
5    $\tilde{v}_i^t \leftarrow \text{FO}(v_i^t, \epsilon_{t,1}, D)$ 
6 Estimate  $\mathbf{r}_{t,1}$  from  $\{\tilde{v}_1^t, \tilde{v}_2^t, \dots, \tilde{v}_N^t\}$ ;
7 Calculate the dissimilarity  $dis_t^*$  by Equation. 7;
    // Sub-mechanism  $\mathcal{M}_{t,2}$ 
8  $\epsilon_{used} \leftarrow 0, \epsilon_{saved} \leftarrow 0$ ;
9 Set budget for each timestamp  $\epsilon_{t,2} = \epsilon/(2w)$ ;
10 if  $\epsilon_{used} = \epsilon/2$  then
11   return  $\mathbf{r}_{t,2} = \mathbf{r}_l$ , set  $\epsilon_{t,2} = 0$ ;
12 else
13   Update  $\epsilon_{t,2} = \epsilon_{t,2} + \epsilon_{saved}$ ;
14   Calculate the publication error  $err$ ;
15   if  $dis_t^* > err_t$  then
16     for each user  $u_i \in U$  do
17        $\tilde{v}_i^t \leftarrow \text{FO}(v_i^t, \epsilon_{t,2}, D)$ 
18     Estimate  $\mathbf{r}_{t,2}$  from  $\{\tilde{v}_1^t, \tilde{v}_2^t, \dots, \tilde{v}_N^t\}$ ;
19     Set  $l = t, \epsilon_{used} \leftarrow \epsilon_{t,2} + \epsilon_{used}$ ;
20     return  $\mathbf{r}_{t,2}$ ;
21   else
22     Set  $\epsilon_{t,2} = 0, \epsilon_{saved} \leftarrow \epsilon_{t,2} + \epsilon_{saved}$ ;
23     return  $\mathbf{r}_{t,2} = \mathbf{r}_l$ ;
24   end if
25 end if
26 end
```

$$\begin{aligned} \text{Var}[\mathbf{r}_{t,1}[j]] &= \mathbb{E}[\mathbf{r}_{t,1}[j] - \mathbf{f}_t[j]]^2 \\ &= \mathbb{E}[(\mathbf{r}_{t,1}[j] - \mathbf{r}_l[j]) - (\mathbf{f}_t[j] - \mathbf{r}_l[j])]^2 \\ &= \mathbb{E}[(\mathbf{r}_{t,1}[j] - \mathbf{r}_l[j])^2 + (\mathbf{f}_t[j] - \mathbf{r}_l[j])^2] \\ &\quad - 2\mathbb{E}[(\mathbf{r}_{t,1}[j] - \mathbf{r}_l[j]) \cdot (\mathbf{f}_t[j] - \mathbf{r}_l[j])] \\ &= \mathbb{E}[\mathbf{r}_{t,1}[j] - \mathbf{r}_l[j]]^2 - (\mathbf{f}_t[j] - \mathbf{r}_l[j])^2. \end{aligned} \quad (9)$$

Based on Eq. 9, we can rewrite $\mathbb{E}[\mathbf{r}_{t,1}[j] - \mathbf{r}_l[j]]^2$ as

$$\mathbb{E}[\mathbf{r}_{t,1}[j] - \mathbf{r}_l[j]]^2 = \text{Var}[\mathbf{r}_{t,1}[j]] + (\mathbf{f}_t[j] - \mathbf{r}_l[j])^2. \quad (10)$$

Combining Equation 8, we can obtain

$$\begin{aligned} & \mathbb{E}[dis_t^*] \\ &= \frac{1}{d} \sum_{j=1}^d ((\mathbf{f}_t[j] - \mathbf{r}_l[j])^2 + \text{Var}[\mathbf{r}_{t,1}[j]]) - \frac{1}{d} \sum_{j=1}^d \text{Var}[\mathbf{r}_{t,1}[j]] \\ &= \frac{1}{d} \sum_{j=1}^d (\mathbf{f}_t[j] - \mathbf{r}_l[j])^2 = dis_t. \end{aligned} \quad (11)$$

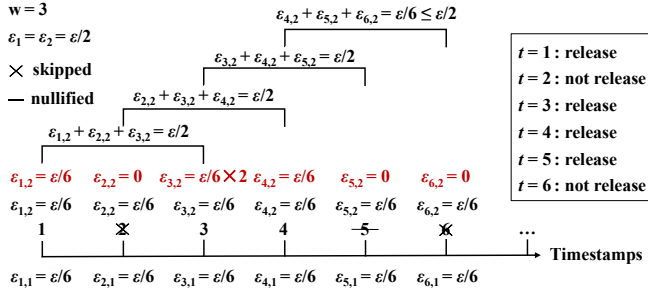


Fig. 3. An example of Fle-BA.

Finally, we prove that dis_t^* is an unbiased estimation of dis_t .

Private publication $\mathcal{M}_{t,2}$: The privacy budget ϵ_2 is also uniformly allocated across all w timestamps for data release (i.e., $\epsilon_{t,2} = \epsilon_2/w$). At each timestamp t , $\mathcal{M}_{t,2}$ must decide whether it is better to approximate the current statistics using the last released statistics or to publish new perturbed statistics. The choice between publication and approximation depends on the relationship between the dissimilarity measure dis_t^* and the publication error err_t . For any noisy statistics $\mathbf{r}_{t,2}$ at timestamp t , because it is an unbiased estimator of the true statistics $\mathbf{f}_t$, its mean squared error equals its variance. Accordingly, the publication error is formulated as $err_t = \frac{1}{d} \sum_{j=1}^d \text{Var}_{\text{FO}}[\mathbf{r}_{t,2}[j]]$.

Based on this formulation, the decision logic at timestamp t proceeds as follows:

- If $dis_t^* > err_t$, the privacy budget $\epsilon_{t,2}$ is used to release the current statistics $\mathbf{r}_{t,2}$.
- Otherwise, the last released timestamp l is used as the approximation of the current timestamp (i.e., $\mathbf{r}_t = \mathbf{r}_l$), and $\epsilon_{t,2} = 0$.

To enhance the utilization of privacy budget, we introduce a flexible budget absorption strategy. If the publication is selected at timestamp t , the corresponding privacy budget is saved; if the approximation is selected, the previously unused budget can be accumulated. Unlike the traditional forced-skip strategy, after the timestamp t has absorbed the budget and completed publication, subsequent timestamps need not be forcibly nullified. This strategy maximizes the efficiency of privacy budget utilization amid uncertain future changes in data. Notably, budgets canceled due to insufficient remaining privacy budget cannot be absorbed in future publications.

Example 2. We will elaborate on the data publication example shown in Figure 2. As illustrated in Figure 3, there are six timestamps with window size $w = 3$. At each timestamp, the sub-mechanisms $\mathcal{M}_{t,1}$ and $\mathcal{M}_{t,2}$ each receive a privacy budget of $\epsilon/(2w) = \epsilon/6$. The sub-mechanism $\mathcal{M}_{t,1}$ uses the perturbed values $\{\tilde{v}_1^t, \tilde{v}_2^t, \dots, \tilde{v}_N^t\}$ at the current timestamp to estimate $\mathbf{r}_{t,1}$, and to calculate the dissimilarity dis_t^* (Lines 3-7 in Algorithm 1). Then, the sub-mechanism $\mathcal{M}_{t,2}$ determines whether to release the current data based on the relationship between dis_t^* and err_t (Lines 15-23 in Algorithm 1).

At timestamp $t = 1$, $\mathcal{M}_{1,2}$ uses budget $\epsilon_{1,2} = \epsilon/6$ to release the statistical result $\mathbf{r}_1$. At timestamp $t = 2$, $\mathcal{M}_{2,2}$ decides not to release the current statistics (skips the current timestamp) and instead uses the most recently released value as an approximation, i.e., $\mathbf{r}_2 = \mathbf{r}_1$. The budget $\epsilon_{2,2}$ is set to 0, and the unused budget $\epsilon/6$ is reserved for absorption at a future timestamp. At timestamp $t = 3$, $\mathcal{M}_{3,2}$ decides to release the current statistics. It absorbs the unused budget $\epsilon/6$ from timestamp 2 and adds it to its own allocated budget, yielding $\epsilon_{3,2} = \epsilon/3$, which is then used to release the statistical result $\mathbf{r}_3$. At timestamp $t = 4$, $\mathcal{M}_{4,2}$ decides to release the current statistics. Within a window of size $w = 3$, the total privacy budget is $\epsilon_{2,2} + \epsilon_{3,2} + \epsilon_{4,2} = 0 + \epsilon/3 + \epsilon/6 = \epsilon/2$. Therefore, the statistical result is released with a budget of $\epsilon/6$ to release $\mathbf{r}_4$. At timestamp $t = 5$, $\mathcal{M}_{5,2}$ decides to release again. However, the total privacy budget within the current window has already reached $\epsilon/2$, i.e., $\epsilon_{3,2} + \epsilon_{4,2} = \epsilon/2$, leaving no remaining budget available for release. Consequently, $\mathcal{M}_{5,2}$ uses the most recently released result as an approximation, i.e., $\mathbf{r}_5 = \mathbf{r}_4$. Furthermore, the nullified privacy budget allocated cannot be utilized for subsequent data releases. At timestamp $t = 6$, $\mathcal{M}_{6,2}$ opts not to release data, with $\epsilon_{6,2} = 0$, and the saved privacy budget is used for subsequent data releases.

Remark. Although adaptive sampling can selectively release data according to stream dynamics, it relies solely on single-point data. It ignores overall data correlations and does not fully utilize correlations across multiple historical timestamps, which may lead to unstable published results. To overcome this limitation, we design a complementary optimization strategy from a completely new perspective that uses multiple historical statistics to capture the evolution pattern of the underlying data. This design provides an effective supplement to the adaptive sampling framework and further improves the utility of long-term private stream publishing.

VI. OUR PROPOSED ADVANCED SCHEME

In this section, we present a novel idea of data grouping over streams under LDP. Then, we propose an adaptive data grouping method for better utility.

A. Technical Overview of AGS

The grouping strategy emphasizes data integrity, which is crucial for capturing transient anomalies and abrupt shifts in distribution. Motivated by this, we construct an Adaptive Grouping Smoothing method consisting of three components: private deviation calculation, data adaptive grouping, and data smoothing, as illustrated in Figure 4.

- **Private Deviation Calculation.** Initially, the total privacy budget ϵ is uniformly allocated across all w timestamps (i.e., $\epsilon_t = \epsilon/w$). N users use the FO protocol to perturb the data and send it to the server. The server then obtains the noisy statistics $\mathbf{r}_t$ at each timestamp t . Subsequently, the server computes the private deviation function $dev^*(G)$, the core measure of similarity between the data.
- **Data Adaptive Grouping.** In AGS, the server compares the deviation function with the perturbation error to determine

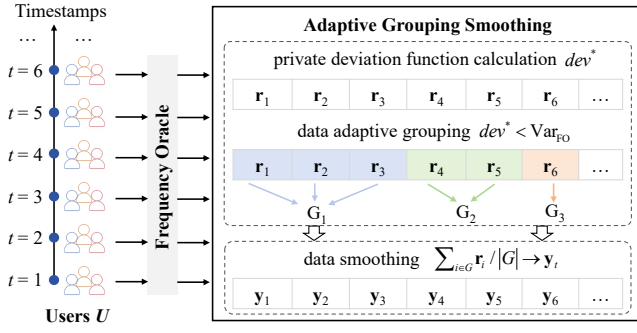


Fig. 4. The workflow of AGS.

whether the current value $\mathbf{r}_t$ should be added to the current open group G_j . At the same time, we record the corresponding partitioning state P_t . The specific usage process of P_t is not illustrated in the figure and will be described in detail in the methodology section.

- **Data Smoothing.** Combining the grouping and partitioning results, we employ an AverageSmoother as a post-processing step. For partition P_t and the last group G at the current timestamp t , the smoothed value is $\mathbf{y}_t = \sum_{i \in G} \mathbf{r}_i / |G|$. Finally, we release the smoothed data sequence.

B. Construction of AGS

We first present the Uniform scheme. Then, we will elaborate on the details of private deviation calculation, data adaptive grouping and data smoothing in AGS.

Strawman: LDP Uniform: To satisfy w -event ϵ -LDP, a naïve approach allocates ϵ evenly across all timestamps in a window of size w . At each timestamp, N users perturb their true values using the FO protocol with budget ϵ/w . The server aggregates the reported results to obtain the estimated frequency $\mathbf{r}_t$ at timestamp t . Therefore, the estimation error over any sliding window of size w can be expressed as:

$$\text{MSE}_{\text{Uniform}} = \frac{1}{w} \sum_{t=1}^{w-1} \text{Var}(\epsilon/w, N) = \text{Var}(\epsilon/w, N). \quad (12)$$

Our approach: Despite the simplicity of the uniform allocation strategy, the estimation error increases as the window size w grows. In real-time data stream publishing, a key challenge is mitigating the impact of injected noise on the data while satisfying privacy guarantees. To this end, we propose *Adaptive Grouping Smoothing (AGS)*, which adaptively groups perturbed values. It tracks the partition status at each timestamp and performs smoothing across multiple historical data points within the current group, thereby significantly reducing error when the data exhibits local stability. In particular, when the data stream exhibits sparse or stable patterns, the grouping method captures these stable regions, while the smoothing strategy leverages the steady-state characteristics within these areas to generate higher-precision estimates.

Private Deviation Calculation. We use the deviation function $dev(G)$, which determines whether the current data should be added to the current group G . The function measures the deviation of the statistical value at the timestamp from its average. Let $\mathbf{f}_1, \mathbf{f}_2, \dots$ be in the group G and $|G|$ be the size of the group G . $dev(G)$ is calculated as follows:

$$dev(G) = \sum_{i \in G} \left(\mathbf{f}_i - \frac{\sum_{i \in G} \mathbf{f}_i}{|G|} \right)^2. \quad (13)$$

However, the server cannot observe the true statistic and thus cannot directly compute $dev(G)$. To overcome this limitation, we leverage the unbiased estimation property and derive a deviation function $dev^*(G)$ that remains unbiased under LDP. The $dev^*(G)$ can be formalized as

$$dev^*(G) = \sum_{i \in G} \left(\mathbf{r}_i - \frac{\sum_{i \in G} \mathbf{r}_i}{|G|} \right)^2 - \left(|G| + \frac{1}{|G|} \right) \text{Var}(\mathbf{r}_i), \quad (14)$$

where $\mathbf{r}_i$ denotes the unbiased estimate of the true statistic $\mathbf{f}_i$. Thus, the expectation of the term $\mathbf{r}_i - \frac{\sum_{i \in G} \mathbf{r}_i}{|G|}$ can also be derived as

$$\begin{aligned} \mathbb{E} \left[\mathbf{r}_i - \frac{\sum_{i \in G} \mathbf{r}_i}{|G|} \right] &= \mathbb{E}[\mathbf{r}_i] - \frac{\sum_{i \in G} \mathbb{E}[\mathbf{r}_i]}{|G|} \\ &= \mathbf{f}_i - \frac{\sum_{i \in G} \mathbf{f}_i}{|G|}. \end{aligned} \quad (15)$$

Then, we can obtain the variance of $\mathbf{r}_i - \frac{\sum_{i \in G} \mathbf{r}_i}{|G|}$

$$\begin{aligned} &\text{Var} \left[\mathbf{r}_i - \frac{\sum_{i \in G} \mathbf{r}_i}{|G|} \right] \\ &= \mathbb{E} \left[\mathbf{r}_i - \frac{\sum_{i \in G} \mathbf{r}_i}{|G|} \right]^2 - \mathbb{E}^2 \left[\mathbf{r}_i - \frac{\sum_{i \in G} \mathbf{r}_i}{|G|} \right] \\ &= \mathbb{E} \left[\mathbf{r}_i - \frac{\sum_{i \in G} \mathbf{r}_i}{|G|} \right]^2 - \left[\mathbf{f}_i - \frac{\sum_{i \in G} \mathbf{f}_i}{|G|} \right]^2. \end{aligned} \quad (16)$$

Based on Equation 15 and 16, we prove that $dev^*(G)$ is an unbiased estimate of $dev(G)$. The expectation is given by

$$\begin{aligned} &\mathbb{E}[dev^*(G)] \\ &= \mathbb{E} \left[dev(G) - \left(|G| + \frac{1}{|G|} \right) \text{Var}(\mathbf{r}_i) \right] \\ &= \mathbb{E} \left[\sum_{i \in G} \left(\mathbf{r}_i - \frac{\sum_{i \in G} \mathbf{r}_i}{|G|} \right)^2 \right] - \left(|G| + \frac{1}{|G|} \right) \text{Var}(\mathbf{r}_i) \\ &= \sum_{i \in G} \left(\mathbf{f}_i - \frac{\sum_{i \in G} \mathbf{f}_i}{|G|} \right)^2 + |G| \text{Var}(\mathbf{r}_i) \\ &\quad + \frac{|G|}{|G|^2} \text{Var}(\mathbf{r}_i) - \left(|G| + \frac{1}{|G|} \right) \text{Var}(\mathbf{r}_i) \\ &= \sum_{i \in G} \left(\mathbf{f}_i - \frac{\sum_{i \in G} \mathbf{f}_i}{|G|} \right)^2 = dev(G). \end{aligned} \quad (17)$$

Algorithm 2: AGS

Input: N users data stream $\{V_1, V_2, \dots, V_N\}$, privacy budget ϵ , window size w , domain D

Output: $\mathcal{Y} = [\mathbf{y}_1, \mathbf{y}_2, \dots]$

```

1 Initialize  $j \leftarrow 1, \mathcal{Y} \leftarrow \emptyset$ ;
2 for each timestamp  $t$  do
3   for each user  $u_i \in U$  do
4      $\tilde{v}_i^t \leftarrow \text{FO}(v_i^t, \epsilon/w, D)$ 
5   Estimate  $\mathbf{r}_t$  from  $\{\tilde{v}_1^t, \tilde{v}_2^t, \dots, \tilde{v}_N^t\}$  with  $\epsilon/w$ ;
6   if  $\text{dev}^*(G_j \cup \mathbf{r}_t) < \text{Var}_{\mathbf{r}_t}$  then
7     Involve  $\mathbf{r}_t$  into  $G_j$  and  $G_j$  still open;
8      $P_t \leftarrow P_{t-1} \cup \{t\}$ ;
9   else
10    Close  $G_j, j \leftarrow j + 1$ ;
11    Involve  $\mathbf{r}_t$  into a new group  $G_j$ ;
12     $P_t \leftarrow \{t\}$ ;
13  end if
14   $\mathbf{y}_t \leftarrow \text{AverageSmoother}(G_j, P_t)$ ;
15  Add  $\mathbf{y}_t$  to  $\mathcal{Y}$ ;
16 end
17 return  $\mathcal{Y}$ .
```

Data Adaptive Grouping. We use $\text{dev}^*(G)$ as the basis for grouping, and the detailed process is presented in Algorithm 2. At each timestamp t , the server first collects private data from N users. These reports are aggregated to compute the current estimated frequency value $\mathbf{r}_t$ (Lines 3-5). Then, at each timestamp t , whether the current data $\mathbf{r}_t$ should be added to the current group G_j is determined by $\text{dev}(G_j \cup \mathbf{r}_t)$ and the theoretical noise variance $\text{Var}_{\mathbf{r}_t}$, where $\text{Var}_{\mathbf{r}_t} = \sum_{k=1}^d \text{Var}_{\text{FO}}[\mathbf{r}_t[j]]$ denotes the sum of variances across d categories.

If the calculated deviation is less than the $\text{Var}_{\mathbf{r}_t}$, $\mathbf{r}_t$ is merged into the current open group G_j , and the partition set is updated to $P_t = P_{t-1} \cup \{t\}$, where P_t records the current grouping status of the latest open group. Otherwise, the current group G_j is closed. The algorithm instantiates a new open group G_{j+1} containing $\mathbf{r}_t$ and resets the partition state to $P_t = \{t\}$ (Lines 6-13). Finally, for each timestamp t , the smoothed value $\mathbf{y}_t$ is calculated based on the current group G_j and the partition P_t (Line 14).

Example 3. The following example illustrates the grouping and partitioning process of AGS. As shown in Figure 4, sliding window $w = 3$, where each user perturbs data with budget $\epsilon/3$. The server estimates a noisy frequency sequence $\mathbf{R} = [\mathbf{r}_1, \mathbf{r}_2, \mathbf{r}_3, \mathbf{r}_4, \mathbf{r}_5, \mathbf{r}_6]$. At timestamp $t = 1$, the server initializes the first group $G_1 = \{\mathbf{r}_1\}$ and the partition $P_1 = \{1\}$. At timestamp $t = 2$, assuming that $\text{dev}(G_1 \cup \mathbf{r}_2) < \text{Var}_{\mathbf{r}_2}$, $\mathbf{r}_2$ is added to the group G_1 and the group remains open. The state updates to $G_1 = \{\mathbf{r}_1, \mathbf{r}_2\}$ and $P_2 = \{1, 2\}$. At timestamp $t = 3$, $\text{dev}(G_1 \cup \mathbf{r}_3) < \text{Var}_{\mathbf{r}_3}$, so $\mathbf{r}_3$ is added to G_1 . Thus, $G_1 = \{\mathbf{r}_1, \mathbf{r}_2, \mathbf{r}_3\}$ and $P_3 = \{1, 2, 3\}$. At timestamp $t = 4$, $\text{dev}(G_1 \cup \mathbf{r}_4) > \text{Var}_{\mathbf{r}_4}$. Therefore, we close G_1 and create a new group G_2 containing $\mathbf{r}_4$. The state updates to $G_2 = \{\mathbf{r}_4\}$ and $P_4 = \{4\}$. At timestamp $t = 5$, $\text{dev}(G_2 \cup \mathbf{r}_5) < \text{Var}_{\mathbf{r}_5}$, so

$\mathbf{r}_5$ is added to G_2 and the group remains open. Thus, $G_2 = \{\mathbf{r}_4, \mathbf{r}_5\}$ and $P_5 = \{4, 5\}$. At timestamp $t = 6$, $\text{dev}(G_2 \cup \mathbf{r}_6) > \text{Var}_{\mathbf{r}_6}$, so we close G_2 and create a new group G_3 . The final state is $G_3 = \{\mathbf{r}_6\}$ and $P_6 = \{6\}$.

Data Smoothing. After collecting the perturbed data, we proceed a smoothing step. Assuming that in partition P_t , the last group is $G_j = \{\mathbf{r}_{t-k}, \dots, \mathbf{r}_{t-1}, \mathbf{r}_t\}$, which contains data of the current time t and the preceding k consecutive timestamps. After obtaining the grouped and partitioned results at the current timestamp, the AverageSmoother is applied to smooth the estimated value $\mathbf{r}_t$. Specifically, the AverageSmoother computes the mean of all noisy values in group G_j to produce the final smoothed result $\mathbf{y}_t$, and is defined

$$\mathbf{y}_t = \frac{\sum_{i \in G} \mathbf{r}_i}{|G_j|}. \quad (18)$$

Example 4. We continue with Example 3 to illustrate the smoothing process executed by the *AverageSmoother*. The noisy values are partitioned into three disjoint groups $G_1 = \{\mathbf{r}_1, \mathbf{r}_2, \mathbf{r}_3\}$, $G_2 = \{\mathbf{r}_4, \mathbf{r}_5\}$, $G_3 = \{\mathbf{r}_6\}$. Accordingly, the partition at each timestamp t is given by $P_1 = \{1\}$, $P_2 = \{1, 2\}$, $P_3 = \{1, 2, 3\}$, $P_4 = \{4\}$, $P_5 = \{4, 5\}$, $P_6 = \{6\}$. Specifically, the smoothed value $\mathbf{y}_t$ is derived by calculating the mean of all noisy values within the current partition P_t . At timestamp $t = 1$, the current partition contains only $\mathbf{y}_1$, $\mathbf{y}_1 = \mathbf{r}_1$. At timestamp $t = 2$, the partition is $P_2 = \{1, 2\}$, $\mathbf{y}_2 = \frac{\mathbf{r}_1 + \mathbf{r}_2}{2}$. At timestamp $t = 3$, the partition is $P_3 = \{1, 2, 3\}$, $\mathbf{y}_3 = \frac{\mathbf{r}_1 + \mathbf{r}_2 + \mathbf{r}_3}{3}$. At timestamp $t = 4$, a new partition $P_4 = \{4\}$ is initiated, $\mathbf{y}_4 = \mathbf{r}_4$. At timestamp $t = 4$, the partition is $P_5 = \{4, 5\}$, $\mathbf{y}_5 = \frac{\mathbf{r}_4 + \mathbf{r}_5}{2}$. Finally, at timestamp $t = 6$, the partition contains only $\mathbf{r}_6$, $\mathbf{y}_6 = \mathbf{r}_6$.

VII. EXTENSION AND DISCUSSION

Mean estimation is also a fundamental and crucial statistical analysis task under LDP. Benefiting from the generality of our framework, it can be naturally extended from frequency estimation to mean estimation. For numerical data streams, we adopt the Piecewise Mechanism (PM) [24] to estimate the mean in our experiments. Specifically, given an input value $v \in [-1, 1]$, the PM perturbs it and outputs a perturbed value $\tilde{v} \in [-C, C]$, where $C = \frac{e^{\epsilon/2} + 1}{e^{\epsilon/2} - 1}$. The probability density function (pdf) of perturbed value $\tilde{v}$ is a piecewise continuous function, and is defined as follows

$$\text{pdf}(\tilde{v} = x|v) = \begin{cases} p, & \text{if } x \in [l(v), r(v)], \\ \frac{p}{e^\epsilon}, & \text{if } x \in [-C, l(v)) \cup (C, r(v)]. \end{cases} \quad (19)$$

where $p = \frac{e^\epsilon - e^{\epsilon/2}}{2e^{\epsilon/2} + 2}$, $l(v) = \frac{C+1}{2}v - \frac{C-1}{2}$, and $r(v) = l(v) + C - 1$.

The mechanism outputs a perturbed value $\tilde{v}$ with $\mathbb{E}[\tilde{v}] = v$, and variance $\frac{v^2}{e^{\epsilon/2}} + \frac{e^{\epsilon/2} + 3}{3(e^{\epsilon/2} - 1)^2}$. Since the input value $v \in [-1, 1]$, the theoretical variance of $\tilde{v}$ can be upper bounded by

$$\text{Var}[\tilde{v}] = \frac{1}{e^{\epsilon/2}} + \frac{e^{\epsilon/2} + 3}{3(e^{\epsilon/2} - 1)^2} = \frac{4e^{\epsilon/2}}{3(e^{\epsilon/2} - 1)^2}. \quad (20)$$

Considering different data generation patterns in real-world applications, we categorize the mean estimation task into the following two scenarios:

- (i) A single user continuously generates a set of numerical data $\{v_1, v_2, \dots\}$, and perturbs each value using PM mechanism. The server estimates the mean from the i -th time slot to the j -th time slot, denoted as $\tilde{m}_{(i,j)} = \frac{\sum_{i=j-i+1}^j \tilde{v}_i}{j-i+1}$.
- (ii) Each user $u_i (i \in N)$ generates a local data stream $V_i = \{v_i^1, v_i^2, \dots\}$. At timestamp t , the server aggregates the perturbed reports to estimate the mean value $\tilde{m}_t = \frac{1}{N} \sum_{i=1}^N \tilde{v}_i^t$.

Both mean estimation scenarios can be naturally incorporated into our framework. In the single-user setting, the dissimilarity dis_t^* is computed from the perturbed value $\tilde{v}$, with variance $\text{Var}[\tilde{v}_t]$. In the multi-user setting, dis_t^* is calculated from the estimated mean $\tilde{m}_t$, and the variance is $\text{Var}[\tilde{m}_t] = \frac{1}{N^2} \sum_{i=1}^N \text{Var}[\tilde{v}_i^t]$.

VIII. THEORETICAL ANALYSIS

In this section, we formally analyze the privacy and utility of the all approaches.

A. Privacy Analysis

In the sliding window framework, the total privacy budget consumed within any window of size w does not exceed ϵ . We prove it in Theorem 8.1.

Theorem 8.1. *Let $\mathcal{M}$ be a mechanism that takes as input a user's stream V_t and output a transcript $o = \{o_1, o_2, \dots, o_t\} \in O$. Suppose that we can decompose $\mathcal{M}$ into t sub-mechanisms $\{\mathcal{M}_1, \mathcal{M}_2, \dots, \mathcal{M}_t\}$, such that $\mathcal{M}_i(V_t[i]) = o_i$ for each $i \in [t]$. If each $\mathcal{M}_i$ generates independent randomness and achieves ϵ_i -LDP, then $\mathcal{M}$ satisfies w -event ϵ -LDP provided that $\sum_{j=i-w+1}^i \epsilon_j \leq \epsilon$.*

Proof. Each sub-mechanism $\mathcal{M}_i$ uses independent randomness, for stream prefix V_t and any output $o = \{o_1, o_2, \dots, o_t\} \in O$, the following equation holds

$$\Pr[\mathcal{M}(V_t) = (o_1, o_2, \dots, o_t)] = \prod_{j=1}^t \Pr[\mathcal{M}_j(V_t[j]) = o_j] \quad (21)$$

Similarly, for any w -neighboring stream prefix V'_t of V_t and the same output $(o_1, o_2, \dots, o_t)$, it holds that

$$\Pr[\mathcal{M}(V'_t) = (o_1, o_2, \dots, o_t)] = \prod_{j=1}^t \Pr[\mathcal{M}_j(V'_t[j]) = o_j] \quad (22)$$

According to the definition of w -neighboring, there exists an index $i \in [t]$ such that $1 \leq j \leq i - w$ and $i + 1 \leq j \leq t$, we have $V_t[j] = V'_t[j]$. Combining this with Equation 21 and 22, we obtain

$$\frac{\Pr[\mathcal{M}(V_t) = (o_1, o_2, \dots, o_t)]}{\Pr[\mathcal{M}(V'_t) = (o_1, o_2, \dots, o_t)]} = \prod_{j=i-w+1}^i \frac{\Pr[\mathcal{M}_j(V_t[j]) = o_j]}{\Pr[\mathcal{M}_j(V'_t[j]) = o_j]}. \quad (23)$$

Due to the Definition 1, each $\mathcal{M}_i$ satisfies ϵ_i -LDP, it follows that $\frac{\Pr[\mathcal{M}_i(V_t[i]) = o_i]}{\Pr[\mathcal{M}_i(V'_t[i]) = o_i]} \leq e^{\epsilon_j}$. Thus, we have

$$\frac{\Pr[\mathcal{M}(V_t) = (o_1, o_2, \dots, o_t)]}{\Pr[\mathcal{M}(V'_t) = (o_1, o_2, \dots, o_t)]} \leq \prod_{j=i-w+1}^i e^{\epsilon_j} = \exp\left(\sum_{j=i-w+1}^i \epsilon_j\right). \quad (24)$$

Since any output $\{o_1, o_2, \dots, o_t\} \in O$, we have $\frac{\Pr[\mathcal{M}(V_t) \in O]}{\Pr[\mathcal{M}(V'_t) \in O]} \leq \exp\left(\sum_{j=i-w+1}^i \epsilon_j\right)$. If formula $\sum_{j=i-w+1}^i \epsilon_k \leq \epsilon$ holds, the probability ration is bound by e^ϵ . That is $\frac{\Pr[\mathcal{M}(V_t) \in O]}{\Pr[\mathcal{M}(V'_t) \in O]} \leq e^\epsilon$. $\square$

We analyze that all approaches satisfy w -event privacy guarantee and the post-processing module does not consume any privacy budget.

Privacy of MPS. Within any sliding window of size w , there are m data publications, each consuming privacy budget $\epsilon_t = \epsilon/m$. For the remaining $w - m$ timestamps, $\epsilon_t = 0$. According to the sequential composition theorem, the overall privacy budget within any sliding window is $\sum_{t=1}^w \epsilon_t = \sum_{t=1}^m \epsilon_t + (w - m) \cdot 0 = m \cdot \epsilon/m = \epsilon$. Therefore, MPS satisfies w -event ϵ -LDP.

Privacy of Fle-BA. We analyze the privacy guarantee of Fle-BA in Theorem 8.2.

Theorem 8.2. *Fle-BA satisfies w -event ϵ -LDP.*

Proof. In sub-mechanism $\mathcal{M}_{t,1}$, the privacy budget $\epsilon_1 = \epsilon/2$ is uniformly allocated across all timestamps within a sliding window of size w . Then, $\mathcal{M}_{t,1}$ is $\epsilon_{t,1}$ -LDP with $\epsilon_{t,1} = \epsilon/2w$. In sub-mechanism $\mathcal{M}_{t,2}$, the privacy budget $\epsilon_2 = \epsilon/2$ for data publication is uniformly allocated across all timestamps, i.e., $\epsilon_{t,2} = \epsilon/2w$. However, privacy budgets are absorbed and publications are nullified, so not every timestamp's data will be published. Therefore, it suffices to prove that $0 \leq \sum_{k=t-w+1}^t \epsilon_{k,2} \leq \epsilon/2$ within a window of size w .

Consider an arbitrary sliding window of size w containing m data publications, s data points that are skipped (with their budgets absorbed), and n nullified publications. The total privacy budget for the current window is $(m + s) \cdot \frac{\epsilon}{2w}$, and the average privacy budget per timestamp is $\epsilon_{t,2} = \frac{(m+s)\epsilon}{2w^2} \leq \frac{\epsilon}{2w}$. Therefore, $\sum_{k=t-w+1}^t \epsilon_{k,2} \leq \sum_{k=t-w+1}^t \epsilon/2w = \epsilon/2$. By the sequential composition theorem, the overall mechanism $\sum_{k=t-w+1}^t \epsilon_{k,1} + \sum_{k=t-w+1}^t \epsilon_{k,2} \leq \epsilon/2 + \epsilon/2 = \epsilon$, which satisfies w -event ϵ -LDP. $\square$

Privacy of AGS. In Algorithm 2, the privacy budget ϵ_t at each timestamp t is ϵ/w . The post-processing AverageS-smoother improves output accuracy without consuming privacy budget. By the sequential composition theorem, the total privacy budget is $\sum_{k=t-w+1}^t \epsilon_k = w \cdot \frac{\epsilon}{w} = \epsilon$, which ensures that AGS satisfies w -event ϵ -LDP.

B. Utility Analysis

In this Section, we analyze the error of released results by all approaches.

Error of MPS. In MPS, m data points are sampled within any sliding window of size w . For each sampled data point, the MSE equals the variance $\text{Var}(\epsilon/m, N)$. For each unsampled data point, the last released $\mathbf{r}_l$ is used as an approximation, and its error expression is $\mathbb{E}[(\mathbf{f}_t - \mathbf{r}_l)^2]$, which can be decomposed into

$$\begin{aligned} & \mathbb{E}[(\mathbf{f}_t - \mathbf{r}_l)^2] \\ &= \mathbb{E}[(\mathbf{f}_t - \mathbf{f}_l) - (\mathbf{r}_l - \mathbf{f}_l)]^2 \\ &= \mathbb{E}[(\mathbf{f}_t - \mathbf{f}_l)^2 - 2(\mathbf{f}_t - \mathbf{f}_l)(\mathbf{r}_l - \mathbf{f}_l) + (\mathbf{r}_l - \mathbf{f}_l)^2] \\ &= (\mathbf{f}_t - \mathbf{f}_l)^2 - 2(\mathbf{f}_t - \mathbf{f}_l)\mathbb{E}(\mathbf{r}_l - \mathbf{f}_l) + \mathbb{E}[(\mathbf{r}_l - \mathbf{f}_l)^2]. \end{aligned} \quad (25)$$

Since $\mathbb{E}[\mathbf{r}_l] = \mathbf{f}_l$, it follows that $\mathbb{E}[\mathbf{r}_l - \mathbf{f}_l] = 0$ and $\mathbb{E}[(\mathbf{r}_l - \mathbf{f}_l)^2] = \text{Var}[\mathbf{r}_l]$. Therefore, Equation 25 can be simplified to

$$\mathbb{E}[(\mathbf{f}_t - \mathbf{r}_l)^2] = (\mathbf{f}_t - \mathbf{f}_l)^2 + \text{Var}[\mathbf{r}_l]. \quad (26)$$

From Equation 26, the MSE for each unsampled data point consists of two components: (i) the noise error, which is the variance of the last released $\mathbf{r}_l$ (i.e., $\text{Var}(\epsilon/m, N)$), and (ii) the approximation error, which is the squared difference (i.e., $(\mathbf{f}_t - \mathbf{f}_l)^2$). The calculation methods of SPS and Uniform are consistent with this approach.

Therefore, the overall error of MPS can be expressed as

$$\begin{aligned} \text{MSE}_{\text{MPS}} &= \frac{1}{w} \left[w \text{Var}(\epsilon/m, N) + \sum_{t=1}^{w-m} (\mathbf{f}_t - \mathbf{f}_l)^2 \right] \\ &= \text{Var}(\epsilon/m, N) + \frac{1}{w} \sum_{t=1}^{w-m} (\mathbf{f}_t - \mathbf{f}_l)^2. \end{aligned} \quad (27)$$

Error of FBA. In FBA, the sub-mechanism $\mathcal{M}_{t,2}$ is used to publish data at each timestamp t . Therefore, the MSE of FBA can be calculated as

$$\begin{aligned} \text{MSE}_{\text{FBA}} &= \frac{1}{w} \left[w \text{Var}(\epsilon_{t,2}, N) + \sum_{t=1}^{w-m} (\mathbf{f}_t - \mathbf{f}_l)^2 \right] \\ &= \text{Var}(\epsilon_{t,2}, N) + \frac{1}{w} \sum_{t=1}^{w-m} (\mathbf{f}_t - \mathbf{f}_l)^2. \end{aligned} \quad (28)$$

In this mechanism, m data points are published, n data points are nullified, and the budget for s timestamps is absorbed, with $s+n = w-m$. Thus, each publication timestamp receives budget $(\frac{s}{m} + 1) \cdot \frac{\epsilon}{2w} = \frac{s+m}{2wm} \cdot \epsilon$, and we have

$$\text{Var}(\epsilon_{t,2}, N) = \text{Var}\left(\frac{s+m}{2wm} \cdot \epsilon, N\right). \quad (29)$$

Error of AGS. In AGS, each data point is perturbed using the FO protocol with budget ϵ/w , we can get

$$\text{MSE}_{\text{AGS}} = \frac{1}{w} \sum_{k=t-w+1}^t \text{Var}(\epsilon/w, N) = \text{Var}(\epsilon/w, N). \quad (30)$$

The theoretical advantages of employing *AverageSmoother* as the post-processing step for AGS are established in the following analysis.

Theorem 8.3. Consider a partition $P = \{t-k, \dots, t-1, t\}$ and the current group $G = \{\mathbf{r}_{t-k}, \dots, \mathbf{r}_{t-1}, \mathbf{r}_t\}$ containing $k \geq 1$ perturbed values. Assume that the released value $\mathbf{y}_t$ at timestamp t is computed using the *AverageSmoother* on the group G . Then, for any timestamp t , it holds that $\text{Var}(\mathbf{y}_t) \leq \text{Var}(\mathbf{r}_t)$.

Proof. By using *AverageSmoother*, the smoothed value $\mathbf{y}_t = \frac{\sum_{i \in G} \mathbf{r}_i}{|G|}$. The expectation of $\mathbf{y}_t$ is formulated as

$$\mathbb{E}[\mathbf{y}_t] = \mathbb{E}\left[\frac{\mathbf{r}_{t-k} + \dots + \mathbf{r}_{t-1} + \mathbf{r}_t}{k}\right]. \quad (31)$$

Then, the variance of $\mathbf{y}_t$ is

$$\text{Var}[\mathbf{y}_t] = \text{Var}\left[\frac{\mathbf{r}_{t-k} + \dots + \mathbf{r}_{t-1} + \mathbf{r}_t}{k}\right]. \quad (32)$$

Due to the linearity of variance, we can expand it as

$$\begin{aligned} \text{Var}[\mathbf{y}_t] &= \frac{1}{k^2} \text{Var}[\mathbf{r}_{t-k} + \dots + \mathbf{r}_{t-1} + \mathbf{r}_t] \\ &= \frac{1}{k^2} (\text{Var}[\mathbf{r}_{t-k}] + \dots + \text{Var}[\mathbf{r}_{t-1}] + \text{Var}[\mathbf{r}_t]). \end{aligned} \quad (33)$$

We consider the variance of all values to be the same as $\text{Var}[\mathbf{r}_t]$, from which we have

$$\text{Var}[\mathbf{y}_t] = \frac{k}{k^2} \text{Var}[\mathbf{r}_t] = \frac{1}{k} \text{Var}[\mathbf{r}_t]. \quad (34)$$

Thus, we have $\text{Var}(\mathbf{y}_t) \leq \text{Var}(\mathbf{r}_t)$, which demonstrates that *AverageSmoother* effectively reduces the variance. $\square$

IX. EXPERIMENTAL EVALUATION

In this section, we conduct extensive experiments to evaluate the effectiveness of our approaches on different datasets.

A. Experimental Setup

Datasets. The experiments are conducted on five real-world datasets and three synthetic datasets.

- **Volume**¹. The dataset was collected from Automatic Traffic Recorder (ATR) station 301, operated by the Minnesota Department of Transportation (MnDOT), and records hourly traffic volumes for this road segment. The data values range from 0 to 7280.
- **Air Quality**². The dataset collected from March 2004 to February 2005, which contains 9,358 air quality instances. We focus on the temporal variation in benzene concentration.
- **Unemployment**³. The dataset records the number of unemployed Black or African American individuals aged 16-19

¹<https://archive.ics.uci.edu/dataset/492/metro+interstate+traffic+volume>

²<https://archive.ics.uci.edu/dataset/360/air+quality>

³<https://fred.stlouisfed.org/series/LNU03000018>

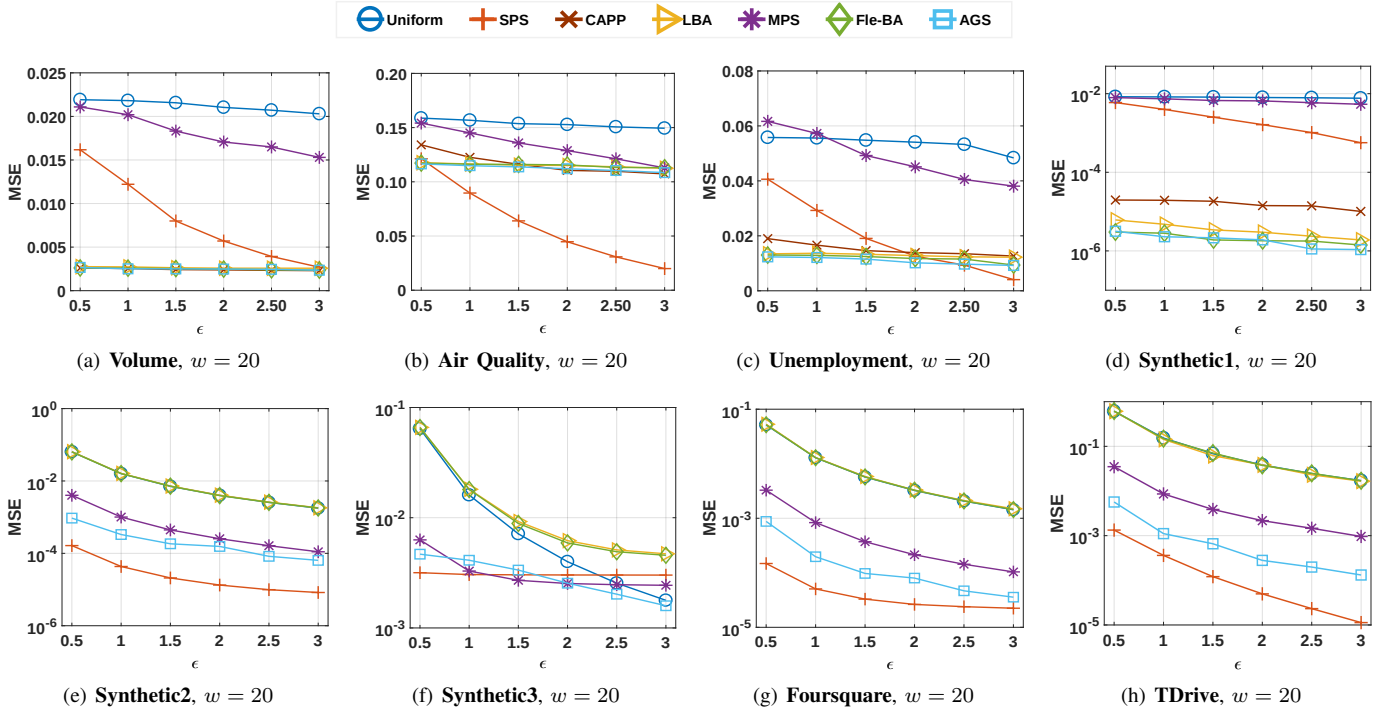


Fig. 5. Comparison of all methods on eight datasets with varying privacy budget ϵ .

in the United States (in thousands). The data values range from 54 to 612.

- **Foursquare**⁴. The dataset contains global-scale check-ins collected from Foursquare between 2012 and 2013 [39]. Each record includes a user ID, timestamp, and location information. We transform it into $N = 122,042$ data streams, each of length $T = 432$, and map check-in locations to $d = 77$ countries.
- **TDrive**⁵. The dataset comprises taxi trajectory data collected in Beijing from February 2 to 8, 2008. We sampled trajectories at 10-minute intervals, yielding $T = 889$ timestamps, and divided the city into 10 spatial grids. Ultimately, we obtained trajectory time series for $N = 10,357$ taxi drivers.

Generation for Synthetic Datasets. We generate three synthetic datasets. For Synthetic1, we simulate a dynamically changing data stream with values in $[1, 1000]$ and a temporal length of $T = 10000$. At each timestamp, the streaming data is randomly sampled from either a Laplace or a Gaussian distribution. The mean and variance of the Laplace and Gaussian distributions are uniformly and randomly sampled from 0 to 99 and 1 to 10, respectively. Synthetic2 contains $N = 100,000$ users with categorical values in $[0, 99]$ and $T = 500$. We simulate a sudden change in the data stream’s distribution: we sample from a uniform distribution over $[0, 99]$ in the first half and from a Gaussian distribution with mean 90 and variance 10 in the second half. Synthetic3 follows the same sampling

strategy as Synthetic1, with $N = 100,000$ users, categorical values in $[0, 99]$, and $T = 500$.

Competitors. We compare our methods with state-of-the-art algorithms for stream releasing under w -event privacy.

- **Uniform**. It adds noise to the data at each timestamp with a uniform privacy budget ϵ/w .
- **SPS**. Single-point sampling method samples only the first timestamp in each window and allocates the entire budget ϵ .
- **CAPP** [16]. A state-of-the-art method for numerical stream release under w -event privacy.
- **LBA** [14]. It exploits temporal variations in data streams and designs a budget absorption strategy. For numerical data, we utilize the PM mechanism in our experiments.

All methods are implemented in Python within a unified experimental framework. The experiments are conducted on a computer with a 2.10 GHz Intel Core i7 CPU and 32 GB RAM.

Metrics. We use Mean Squared Error (MSE) and Mean Absolute Error (MAE) as accuracy metrics, measuring the difference between the released results $\mathbf{R} = [\mathbf{r}_1, \mathbf{r}_2, \dots]$ and the true statistics $\mathbf{F} = [\mathbf{f}_1, \mathbf{f}_2, \dots]$. Let $\mathbf{r}_t = [\mathbf{r}_t[1], \mathbf{r}_t[2], \dots, \mathbf{r}_t[d]]$ denote the released result at the t^{th} timestamp, and let d denote the domain size.

- **MSE**. MSE measures the average squared difference between the released results and true statistics. Specifically, for the released result $\mathbf{r}_t$, the MSE is calculated as:

$$\text{MSE} = \frac{1}{d} \sum_{j=1}^d (\mathbf{r}_t[j] - \mathbf{f}_t[j])^2. \quad (35)$$

⁴<https://sites.google.com/site/yangdingqi/home/publication>

⁵<https://www.microsoft.com/en-us/research/publication/t-drive-trajectory-data-sample>

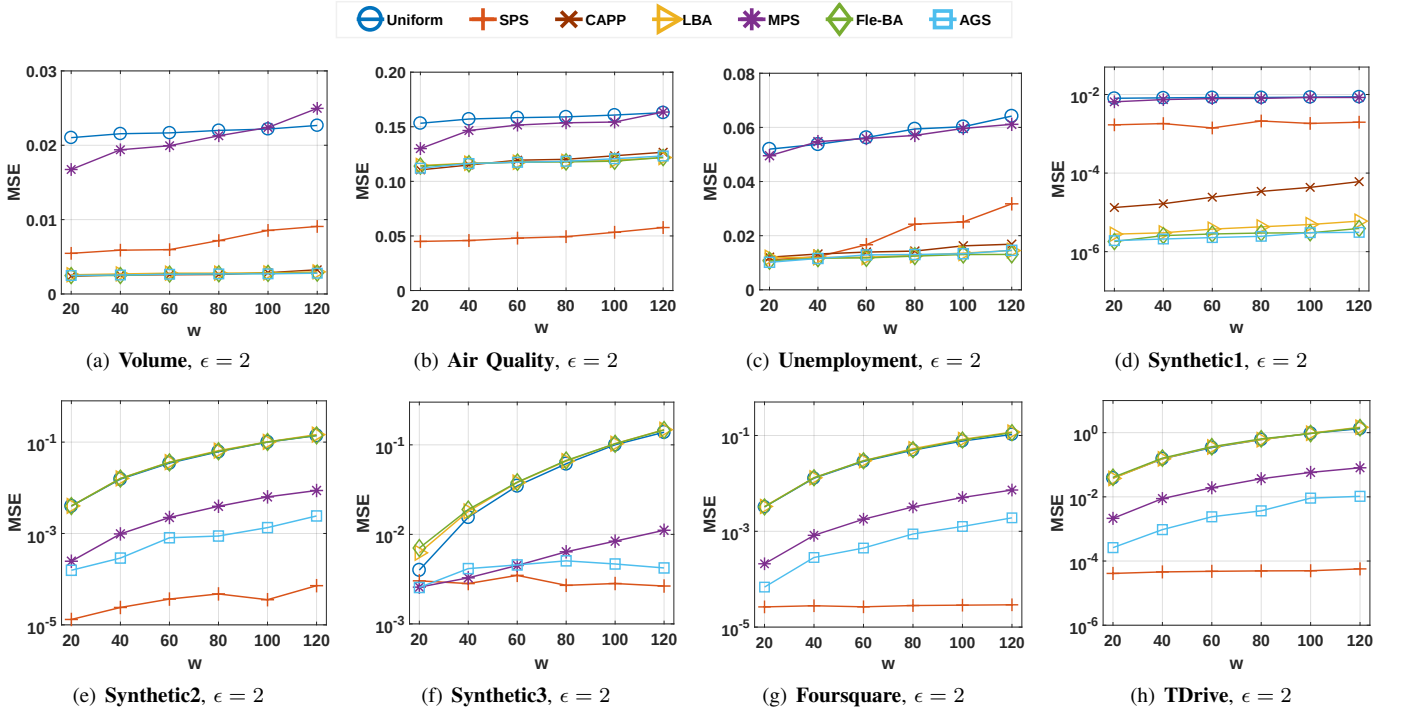


Fig. 6. Comparison of all methods on eight datasets with varying sliding window size w .

TABLE II
MEAN ABSOLUTE ERROR (MAE) OF MPS METHOD UNDER DIFFERENT PARAMETER SETTINGS

Dataset	$\epsilon = 2, w = 20$				$\epsilon = 4, w = 20$				$\epsilon = 4, w = 80$			
	$m = 2$	$m = 4$	$m = 8$	$m = 16$	$m = 2$	$m = 4$	$m = 8$	$m = 16$	$m = 8$	$m = 16$	$m = 32$	$m = 64$
Volume	0.1075	0.1281	0.1378	0.1436	0.0765	0.1069	0.1268	0.1376	0.1262	0.1379	0.1437	0.1461
Synthetic1	0.0659	0.0798	0.0872	0.0915	0.0429	0.0659	0.0789	0.0874	0.0813	0.0871	0.0914	0.0933
Synthetic2	0.0049	0.0101	0.0199	0.0401	0.0024	0.0049	0.0099	0.0209	0.0102	0.0197	0.0101	0.0804
Synthetic3	0.0204	0.0228	0.0288	0.0425	0.0185	0.0187	0.0197	0.0229	0.0247	0.0316	0.0472	0.0821
Foursquare	0.0051	0.0094	0.0182	0.0361	0.0031	0.0052	0.0093	0.0181	0.0095	0.0181	0.0367	0.0724
Tdrive	0.0161	0.0311	0.0591	0.1187	0.0051	0.0155	0.0296	0.0597	0.0304	0.0609	0.1172	0.2395

- **MAE.** MAE measures the average absolute error between the released results and true statistics. In particular, for the released result $\mathbf{r}_t$ timestamp, the MSE is calculated as:

$$\text{MAE} = \frac{1}{d} \sum_{j=1}^d |\mathbf{r}_t[j] - \mathbf{f}_t[j]|. \quad (36)$$

B. Experimental Results

To validate the effectiveness of our methods, we conduct comparative accuracy analysis under different parameter settings. Since different methods exhibit varying sensitivities to the privacy budget ϵ and window size w , non-monotonic behavior trends may be observed on certain datasets. Next, we will analyze the experimental results in detail.

1) **Comparison of Privacy Budget ϵ :** In Figure 5, we vary the privacy budget ϵ from 0.5 to 3 and compare the MSE across all methods on eight datasets. The sliding window size w is fixed at 20, and the parameter m is set to 5 in MPS.

For single-user datasets, we estimate the mean of numerical data. The experimental results in Figure 5(a)-5(d) show that the estimation error of all methods decreases as ϵ increases. SPS performs best on Volume, Air Quality, and Unemployment datasets, all of which come from real-world scenarios with relatively stable data distributions. In such case, sampling a single timestamp is sufficient to capture the overall statistics within the sliding window. In contrast, AGS exhibits excellent performance on Synthetic1 dataset, while SPS fails to capture critical timestamps.

For multi-user datasets, frequency estimation was performed on categorical data, as shown in Figure 5(e)-5(h). Fle-BA improves the budget absorption strategy, but its performance improvement is relatively limited compared to LBA. As ϵ increases, SPS performs poorly on Synthetic3 and Foursquare datasets, indicating that sampling a single timestamp cannot effectively capture complex temporal dynamics. Benefiting from multi-timestamp sampling, MPS achieves performance

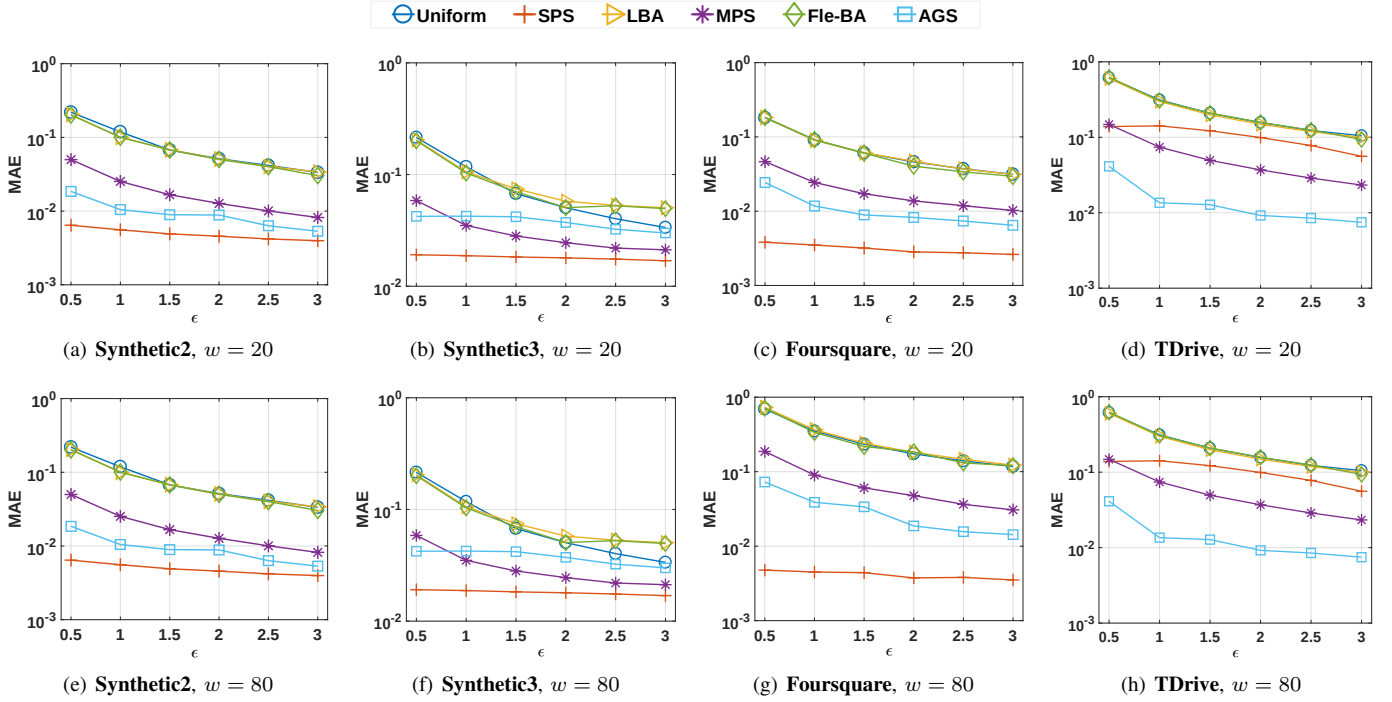


Fig. 7. Range query evaluation with varying privacy budget ϵ and sliding window size w .

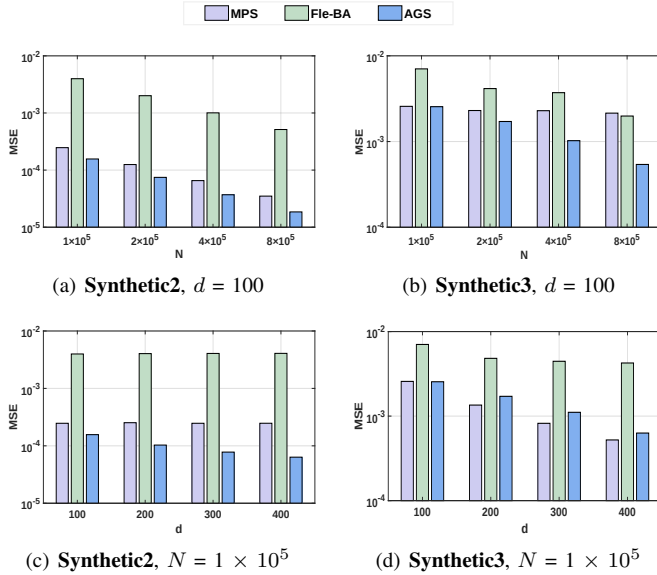


Fig. 8. Effect of population size N and domain size d .

between Uniform and SPS, while gradually narrowing the gap with SPS. Notably, AGS consistently performs well across all

2) **Comparison of Window Size w :** In Figure 6, we examine the accuracy of various w -event ϵ -LDP methods as the window size w varies from 20 to 120, with the budget fixed at $\epsilon = 2$. Overall, as w increases, the MSE across all methods increases. For sample-based methods, the single-point data release strategy cannot fully capture the dynamics within each window. Although the multi-point sampling strategy adjusts

the sampling parameter $m \in \{5, 10, 15, 20, 25, 30\}$ as w increases, random sampling still struggles to accurately represent the trend of data variation. In contrast, Fle-BA adaptively releases data based on the dissimilarity of historical information, better capturing data stream dynamics. Consequently, its error exhibits a gentler upward trend as w grows. For the data group-based method, as w increases, the privacy budget allocated at each time point decreases, leading to larger estimation errors. However, AGS uses multiple historical data points to smooth the results at the current timestamp, effectively mitigating performance degradation caused by budget dilution.

3) **Varying Sampling Number m :** In Table II, we vary m under a fixed window size w to examine its impact on MPS performance. Under a fixed budget $\epsilon = 2$, the results on all six datasets consistently show that the overall error increases significantly as m grows. This is because the total privacy budget is split across m sampling points. Consequently, the effective budget of ϵ/m at each point introduces stronger noise and increases the total error. As ϵ increases, the overall estimation error consistently decreases. Furthermore, the estimation error exhibits a monotonic increase as m grows under $w = 80$. Notably, on the Synthetic3 dataset, the error does not show a significant upward trend as m increases. This is primarily because the data is randomly sampled from either a Laplace or a Gaussian distribution at each timestamp, resulting in an overall unstable distribution that lacks persistent patterns of change. Under such conditions, increasing m does not substantially amplify the error. This observation indicates that the optimal choice of m is closely tied to the distributional characteristics of the dataset.

Overall, the experimental results show that although a larger m introduces more noise due to privacy budget splitting, it has the advantage of capturing data changes as accurately as possible and reducing cumulative errors. Consequently, in practical applications, it is necessary to select m based on the data's variability and precision requirements to balance data utility and privacy protection.

4) **Evaluation for Range Query:** We further evaluate the performance of the released data streams from six competitors on range queries. Specifically, we randomly generate 000 equal-length range queries, each corresponding to an interval $[v_1, v_2]$, where both v_1 and v_2 are below the upper bound of the data domain. The answer to each query is the number of users whose values fall within the specified range across all timestamps. We compare all methods on two real-world and two synthetic datasets and report the average MAE across all query results. The experimental results are shown in Figure 7.

In Figures 7(a)-7(d), the window size is fixed at $w = 20$, while in Figures 7(e)-7(h), it is fixed at $w = 80$. As w increases, the MAE across all methods generally increases, indicating that a larger sliding window leads to greater accumulated estimation error. Compared with the existing LBA method, AGS consistently achieves superior performance across datasets and window sizes, particularly on the TDrive dataset, demonstrating its effectiveness in improving the utility of range-query results.

5) **Effect of Dataset Parameters:** To investigate the impact of key dataset parameters on utility, we vary the population size N and the domain size d on the Synthetic2 and Synthetic3 datasets, respectively. When varying N , we keep d fixed. As shown in Figures 8(a)-8(b), the MSE of all methods decreases monotonically as N increases. This is because, under a fixed domain size d , a larger population improves the accuracy of statistical results. Figures 8(c)-8(d) further illustrate the effect of domain size d on the experimental results. With a fixed population N , the methods exhibit noticeably different trends as d increases. Specifically, the timestamp length remains unchanged, and the performance of MPS is not significantly affected. Its overall error remains relatively stable. In contrast, Fle-BA determines whether to release the current statistics based on perturbation error, which is highly correlated with the domain size and thus affects release decisions. Moreover, as d grows, the underlying frequency distribution changes. AGS can capture change patterns between adjacent data points, thereby maintaining superior performance even at larger domain scales.

X. CONCLUSION

In this paper, we focus on stream data collection and release under the w -event LDP and propose a set of novel and efficient methods. From sampling perspective, we propose the fixed threshold sampling method MPS and the adaptive data sampling method Fle-BA. From data grouping perspective, we design AGS based on the dynamic partitioning and smoothing of perturbed data. We rigorously prove that all methods satisfy w -event ϵ -LDP and provide corresponding utility analyses.

Extensive experimental results demonstrate the effectiveness of our methods. In future work, we plan to develop more fine-grained data dependent release strategies and extend our methods to more complex data stream analysis tasks and emerging application scenarios.

ACKNOWLEDGMENT

This work was supported by the National Key Research and Development Program of China (No. 2024YFB3108700), National Cryptologic Science Fund of China (2025NCSF02031), Fundamental Research Funds for the Central Universities (No. QTZX24077), Shaanxi Provincial Natural Science Foundation Project (No. 2025JC-TBZC-12), National Natural Science Foundation of China (No. 62125205, No. U23A20303, No. 62372493), and the Open Fund of Key Laboratory of Computing Power Network and Information Security (No. 2023ZD020).

REFERENCES

- [1] J. L. Ny, A. Touati, and G. J. Pappas, "Real-time privacy-preserving model-based estimation of traffic flows," in *2014 ACM/IEEE International Conference on Cyber-Physical Systems (ICCPs)*, 2014, pp. 92–102.
- [2] Y. Zheng, L. Capra, O. Wolfson, and H. Yang, "Urban computing: Concepts, methodologies, and applications," *ACM Transactions on Intelligent Systems and Technology (TIST)*, vol. 5, no. 3, pp. 1–55, 2014.
- [3] M. E. Edge, P. R. F. Sampaio, and M. Choudhary, "Towards a proactive fraud management framework for financial data streams," in *Third IEEE International Symposium on Dependable, Autonomic and Secure Computing (DASC)*, 2007, pp. 55–64.
- [4] J. C. Duchi, M. I. Jordan, and M. J. Wainwright, "Local privacy and statistical minimax rates," in *2013 IEEE 54th Annual Symposium on Foundations of Computer Science (FOCS)*, 2013.
- [5] S. P. Kasiviswanathan, H. K. Lee, K. Nissim, S. Raskhodnikova, and A. Smith, "What can we learn privately?" *SIAM Journal on Computing*, vol. 40, no. 3, pp. 793–826, 2011.
- [6] U. Erlingsson, V. Pihur, and A. Korolova, "Rappor: Randomized aggregatable privacy-preserving ordinal response," in *Proceedings of the 2014 ACM SIGSAC Conference on Computer and Communications Security (CCS)*, 2014, pp. 1054–1067.
- [7] A. Differential Privacy Team, "Learning with privacy at scale," 2017. [Online]. Available: <https://machinelearning.apple.com/research/learning-with-privacy-at-scale>
- [8] B. Ding, J. Kulkarni, and S. Yekhanin, "Collecting telemetry data privately," in *Proceedings of the 31st International Conference on Neural Information Processing Systems (NIPS)*, 2017, pp. 3574–3583.
- [9] C. Dwork, M. Naor, T. Pitassi, and G. N. Rothblum, "Differential privacy under continual observation," 2010, p. 715–724.
- [10] V. Perrier, H. J. Asghar, and D. Kaafar, "Private continual release of real-valued data streams," *Proceedings 2019 Network and Distributed System Security Symposium (NDSS)*, 2019.
- [11] L. Fan and L. Xiong, "Real-time aggregate monitoring with differential privacy," in *Proceedings of the 21st ACM International Conference on Information and Knowledge Management (CIKM)*, 2012, p. 2169–2173.
- [12] E. Bao, Y. Yang, X. Xiao, and B. Ding, "Cgm: an enhanced mechanism for streaming data collection with local differential privacy," *Proceedings of the VLDB Endowment*, vol. 14, pp. 2258–2270, 2021.
- [13] G. Kellaris, S. Papadopoulos, X. Xiao, and D. Papadias, "Differentially private event sequences over infinite streams," *Proceedings of the VLDB Endowment*, vol. 7, no. 12, pp. 1155–1166, 2014.
- [14] X. Ren, L. Shi, W. Yu, S. Yang, C. Zhao, and Z. Xu, "Ldp-ids: Local differential privacy for infinite data streams," in *Proceedings of the 2022 International Conference on Management of Data (SIGMOD)*, 2022, p. 1064–1077.
- [15] Y. Hu, Y. Du, Z. Zhang, Z. Fang, L. Chen, K. Zheng, and Y. Gao, "Real-time trajectory synthesis with local differential privacy," in *2024 IEEE 40th International Conference on Data Engineering (ICDE)*, 2024, pp. 1685–1698.

- [16] R. Du, Q. Ye, Y. Xiao, L. Yu, Y. Fu, and H. Hu, "Dual utilization of perturbation for stream data publication under local differential privacy," in *2025 IEEE 41st International Conference on Data Engineering (ICDE)*, 2025, pp. 3522–3534.
- [17] Z. Wang, W. Liu, X. Pang, J. Ren, Z. Liu, and Y. Chen, "Towards pattern-aware privacy-preserving real-time data collection," in *IEEE INFOCOM 2020 - IEEE Conference on Computer Communications*, 2020, pp. 109–118.
- [18] C. Dwork, "Differential privacy," in *ICALP*. Springer, 2006, pp. 1–12.
- [19] C. Dwork, F. McSherry, K. Nissim, and A. Smith, "Calibrating noise to sensitivity in private data analysis," in *Proceedings of the Third Conference on Theory of Cryptography (TCC)*, 2006, pp. 265–284.
- [20] J. Acharya, Z. Sun, and H. Zhang, "Hadamard response: Estimating distributions privately, efficiently, and with little communication," vol. 89, 2019, pp. 1120–1129.
- [21] R. Bassily and A. Smith, "Local, private, efficient protocols for succinct histograms," 2015, pp. 127–135.
- [22] T. Wang, J. Blocki, N. Li, and S. Jha, "Locally differentially private protocols for frequency estimation," in *Proceedings of the 26th USENIX Conference on Security Symposium*, 2017, pp. 729–745.
- [23] Z. Li, T. Wang, M. Lohpää-Zwakenberg, N. Li, and B. Škoric, "Estimating numerical distributions under local differential privacy," in *Proceedings of the 2020 ACM SIGMOD International Conference on Management of Data*, 2020, pp. 621–635.
- [24] N. Wang, X. Xiao, Y. Yang, J. Zhao, S. C. Hui, H. Shin, J. Shin, and G. Yu, "Collecting and analyzing multidimensional data with local differential privacy," in *2019 IEEE 35th International Conference on Data Engineering (ICDE)*, 2019, pp. 638–649.
- [25] S. Xu, S. Su, L. Xiong, X. Cheng, and K. Xiao, "Differentially private frequent subgraph mining," in *2016 IEEE 32nd International Conference on Data Engineering (ICDE)*, 2016, pp. 229–240.
- [26] X. Cheng, S. Su, S. Xu, L. Xiong, K. Xiao, and M. Zhao, "A two-phase algorithm for differentially private frequent subgraph mining," *IEEE Transactions on Knowledge and Data Engineering (TKDE)*, vol. 30, no. 8, pp. 1411–1425, 2018.
- [27] T. Wang, J. Q. Chen, Z. Zhang, D. Su, Y. Cheng, Z. Li, N. Li, and S. Jha, "Continuous release of data streams under both centralized and local differential privacy," in *Proceedings of the 2021 ACM SIGSAC Conference on Computer and Communications Security (CCS)*, 2021, p. 1237–1253.
- [28] Y. Chen, A. Machanavajjhala, M. Hay, and G. Miklau, "Pegasus: Data-adaptive differentially private stream processing," in *Proceedings of the 2017 ACM SIGSAC Conference on Computer and Communications Security (CCS)*, 2017, p. 1375–1388.
- [29] X. Li, Z. Qin, K. Ren, C. Gong, S. Feng, Y. Hong, and T. Wang, "Delay-allowed differentially private data stream release," *Proceedings 2025 Network and Distributed System Security Symposium (NDSS)*, 2025.
- [30] Y. Mao, Q. Ye, H. Hu, Q. Wang, and K. Huang, "Privshape: Extracting shapes in time series under user-level local differential privacy," 2024, pp. 1739–1751.
- [31] S. Feng, M. Mohammady, H. Wang, X. Li, Z. Qin, and Y. Hong, "Dpi: Ensuring strict differential privacy for infinite data streaming," in *2024 IEEE Symposium on Security and Privacy (SP)*, 2024, pp. 1009–1027.
- [32] H. Li, L. Xiong, X. Jiang, and J. Liu, "Differentially private histogram publication for dynamic datasets: an adaptive sampling approach," in *Proceedings of the 24th ACM International Conference on Information and Knowledge Management (CIKM)*, 2015, pp. 1001–1010.
- [33] T. Wang, X. Yang, X. Ren, J. Zhao, and K.-Y. Lam, "Adaptive differentially private data stream publishing in spatio-temporal monitoring of iot," in *2019 IEEE 38th International Performance Computing and Communications Conference (IPCCC)*, 2019, pp. 1–8.
- [34] X. Li, T. Li, Y. Cheng, C. Gong, K. Ren, Z. Qin, and T. Wang, "Spas: Continuous release of data streams under w-event differential privacy," *Proceedings of the ACM on Management of Data (SIGMOD)*, vol. 3, no. 1, pp. 1–27, 2025.
- [35] L. Du, P. Cheng, L. Chen, H. T. Shen, X. Lin, and W. Xi, "Infinite stream estimation under personalized w-event privacy," *Proceedings of the VLDB Endowment*, vol. 18, no. 6, pp. 1905–1918, 2025.
- [36] C. Dwork and A. Roth, "The algorithmic foundations of differential privacy," *Foundations and Trends® in Theoretical Computer Science*, vol. 9, pp. 211–407, 2014.
- [37] P. Kairouz, S. Oh, and P. Viswanath, "Extremal mechanisms for local differential privacy," *Journal of Machine Learning Research*, vol. 17, no. 1, pp. 492–542, 2016.
- [38] S. L. Warner, "Randomized response: a survey technique for eliminating evasive answer bias," *Journal of the American Statistical Association*, vol. 60(309), pp. 63–69, 1965.
- [39] D. Yang, B. Qu, J. Yang, and P. Cudre-Mauroux, "Revisiting user mobility and social relationships in lbsns: A hypergraph embedding approach," in *The World Wide Web Conference (WWW)*, 2019, pp. 2147–2157.